

Deceptive-Net: A Honey-Token Generative Framework

A PROJECT REPORT

**Submitted in partial fulfilment of the
Requirement for the award of the degree
of**

MASTER OF COMPUTER APPLICATIONS (MCA)

by

Manoj Kumar Sharma

2424150118



**MANIPAL UNIVERSITY
JAIPUR**

DEPARTMENT OF COMPUTER APPLICATIONS

MANIPAL UNIVERSITY JAIPUR

JAIPUR-303007

RAJASTHAN, INDIA

May 2026

DEPARTMENT OF COMPUTER APPLICATIONS
MANIPAL UNIVERSITY JAIPUR, JAIPUR-303 007 (RAJASTHAN),
INDIA

Date:

CERTIFICATE

This is to certify that the project titled **DECEPTIVE-NET: A HONEY-TOKEN GENERATION FRAMEWORK** is a record of the Bonafide work done by **MANOJ KUMAR SHARMA (2424150118)** submitted in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications (MCA) of Manipal University Jaipur, during the academic year 2025-26.

Dr. Honey Gocher
Project Guide, Department of Computer Applications
Manipal University Jaipur

Dr. Shilpa Sharma
HOD, Department of Computer Applications
Manipal University Jaipur

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to the Dean and Director of Manipal University Jaipur for providing the necessary infrastructure and academic environment that facilitated this project work. I extend my heartfelt thanks to the Head of the Department of Computer Science and Engineering for the constant encouragement and support throughout the duration of this project.

I am deeply indebted to my Project Guide, whose invaluable guidance, timely feedback, and unwavering support were instrumental in shaping this project from conception to completion. The expert advice, constructive criticism, and scholarly insights offered at every stage of the development process have been truly enriching.

I am sincerely grateful to my industry supervisor and the team at the organization where this project was executed, for providing the practical exposure, technical resources, and a productive working environment. Their mentorship helped bridge the gap between theoretical knowledge and real-world application development.

I extend my appreciation to all the faculty members of the Department of Computer Applications whose lectures, discussions, and guidance throughout the MCA programme laid the strong conceptual foundation required for this project. Special thanks to the laboratory staff for their technical support during testing and demonstration phases.

I owe a deep sense of gratitude to my family and friends for their constant encouragement, patience, and moral support during the demanding phases of this project work. Without their backing, this endeavour would not have been possible.

ABSTRACT

In today's cyber landscape, where attackers are constantly finding new ways to bypass traditional signature-based and anomaly-detection security measures, cyber deception is becoming a pivotal proactive defence tactic in information security. In the case of honey-tokens, the attacker takes advantage of his or her own trust in the compromised credential store by marking a malicious activity when the token is being accessed or exfiltrated. The problem with the static, hand-made honey-tokens, however, is believability – there is very little value in these honey-tokens when highly skilled threat actors can easily fingerprint them. This project aims to overcome this drawback by creating Deceptive-Net++, a causally-aligned, neuro-symbolic generative framework for generating autonomous honey-tokens and their adaptive deployment in live honeypot environments.

In this project, five tightly coupled technical modules are combined in the methodology. A Fraud Dependency Graph (FDG) is a learned causal graph over the NeurIPS 2022 Bank Account Fraud (BAF) dataset that is used to capture the causal structure in the generation process. He trains this FDG model to jointly generate behaviorally plausible tabular honey-tokens, based on both the financial features in BAF and Mendeley Synthetic PII (MSPII) records, using a Conditional Wasserstein GAN with Gradient Penalty (CWGAN-GP) together with Feature-wise Linear Modulation (FiLM) and a dual-domain multi-head critic. A Credential-Aware Attention (CAA) module introduces logical semantic constraints: Luhn credit-card numbers, zip-to-state geographical consistency, and username-to-email semantic consistency, using a differentiable constraint-preservation loss. Lastly, a Deep Q-Network (DQN) Tactical Deployment Agent (TDA) uses a Markov Decision Process (MDP) over the logs of interactions with a Cowrie honeypot to adaptively select and update deployment strategies according to live attacker behaviour.

The experimental results show that the method achieves state-of-the-art results in terms of all the evaluation metrics. Deceptive-Net++ obtains a Deception Discriminability Score (DDS) of 0.669, which is 16 times less than the best comparable baseline (SMOTE-NC: 1.423), showing that tokens generated by this algorithm are almost as similar to real ones as is possible statistically. The framework achieves highest Luhn-valid credit-card generation of 35.9% and name-email semantic coherence of 90.7% compared to all previous frameworks. Deceptive-Net++ improved median attacker dwell time from 1.6 minutes to 26 minutes (16.25x), exfiltration-attempt events (from 14 to 231 events, a 16.5x increase) and synthetic-token detection (from 9% to 1%). An exhaustive ablation study using 8 configurations validates the independent and significant contributions of each proposed component, where the largest ablation impact (+75%) comes from the FiLM conditioning, record alignment (+47%) and the TDA (+29%).

The project utilized Python 3.10, PyTorch 2.1, PyTorch Geometric for graph neural network operations and the Cowrie SSH/Telnet honeypot framework for real deployment. All the experiments performed for the training and inference were done using a single NVIDIA GTX 1650 Ti (4 GB GDDR6) GPU, showing the applicability of the framework on commodity consumer devices. Two new evaluation metrics, called Deception Discriminability Score (DDS) and Deception Realism Score (DRS) are presented as community resources for the standardisation of evaluation of the honey-token generation system. A public 10,000 sample

honey-tokens dataset is provided together with the evaluation metric code and training scripts.

Keywords: Cyber deception, honey-tokens, honeypot systems, Deceptive-Net++, neuro-symbolic security, autonomous honey-token generation, adaptive cyber deception, Fraud Dependency Graph (FDG), Conditional Wasserstein GAN with Gradient Penalty (CWGAN-GP), Feature-wise Linear Modulation (FiLM), dual-domain multi-head critic, Credential-Aware Attention (CAA), semantic constraint preservation, Deep Q-Network (DQN), Tactical Deployment Agent (TDA), attacker dwell time, deception realism, deception discriminability, synthetic credential generation, cyber threat intelligence, network security, adversarial defence, Cowrie honeypot, graph neural networks, generative adversarial networks, reinforcement learning, tabular data synthesis, financial fraud datasets, synthetic personally identifiable information (PII), deception deployment strategies, behavioural plausibility, cyber defence automation, semantic coherence, Luhn validation, live honeypot deployment, attacker interaction analysis, explainable AI security, PyTorch, PyTorch Geometric, intelligent deception systems, cybersecurity research, deception realism score (DRS), deception discriminability score (DDS)

LIST OF TABLES

Table No	Table Title	Page No
Table 1.1	Operating Environment	2
Table 2.1	Key Technologies in Deceptive Net	7
Table 3.1	Operating Environment	8
Table 3.2	Software and Hardware Requirement Specifications	8
Table 3.3	BAF Dataset Feature Group Summary	12
Table 3.4	Mendeley Synthetic PII Domain Summary	12
Table 3.5	Data Dictionary	12
Table 4.1	Experimental Setup	23
Table 4.2	Main Comparison Results	23
Table 4.3	Ablation Study Results	25
Table 4.4	Live Cowrie Honeypot Deployment Results	26
Table 4.5	System Profiling Summary	27
Table 5.1	Dataset Role in the Deceptive-Net Pipeline	35
Table 5.2	FDG Design Elements	35
Table 5.3	Generator and Critic Hyperparameters	36
Table 5.4	Credentials-Aware Constraint Terms	37
Table 5.5	MDP Formulation for TDA	37
Table 5.6	Baseline Interpretation	38
Table A.1	Sample Generated Honey-Token Records	46
Table B.1	BAF Dataset Feature Groups	46
Table C.1	DQN TDA State Action Reward Specification	47

LIST OF FIGURES

Figure No	Figure Title	Page No
Figure 3.1	Context-Level DFD for Deceptive-Net	10
Figure 3.2	Level-1 Data Flow Diagram	11
Figure 3.3	Functional Decomposition Diagram	12
Figure 3.4	Entity Relationship View	12
Figure 4.1	Deceptive-Net System Architecture	23
Figure 4.2	Training Dynamic Figure	23
Figure 4.3	t-SNE Distributional Comparison	23
Figure 4.4	DDS Comparison Plot	24
Figure 4.5	Luhn Validity Plot	25
Figure 4.6	Ablation Study Plot	26
Figure 4.7	Honeypot Deployment Plot	28
Figure 4.8	Generation Benchmark Visualization	30
Figure 4.9	DDS Comparison Visualization	30
Figure 4.10	Confusion Matrix Style Result View	31
Figure 4.11	ROC Curve Style Result View	32
Figure 4.12	Precision Recall Style Result View	32
Figure 4.13	Feature Importance Style View	33
Figure 4.14	Scenario Ablation Visualization	33
Figure 4.15	Runtime Profiling Visualization	33
Figure 4.16	TDA Engagement Visaulization	34

TABLE OF CONTENTS

	Page No
Acknowledgement	i
Abstract	ii
List Of Figures	iii
List Of Tables	iv
Chapter 1	INTRODUCTION
1.1	Project Overview
1.2	Existing System and Need for System
1.3	Scope of Work
1.4	Operating Environment
1.5	Related Work Summary
1.6	Threat Model
Chapter 2	BACKGROUND MATERIAL
2.1	Honey-Token and Honeypot Systems
2.2	Tabular Generative Models
2.3	Conditional GANs and WGAN-GP
2.4	Casual Generative Models
2.5	Reinforcement Learning for Security
2.6	Technologies Involved
Chapter 3	SYSTEM ANALYSIS AND DESIGN
3.1	Project Overview
3.2	Software and Hardware Requirement Specifications
3.3	Data Flow Diagram
3.4	Functional Decomposition Diagram
3.5	Entity Relationship Diagram ERD
3.6	Data Dictionary and Table Design
3.7	Detailed Architecture Design
3.8	Evaluation Matric Design
3.9	Code Design
3.10	Menu Tree and Screen Design Consideration
Chapter 4	TEST PROCEDURES AND IMPLEMENTATION
3.1	Project Code and Algorithmic Snippets
3.2	Screenshots and Project Figures
3.3	Experimental Setup and Results
3.4	Implementation and Deployment Details
3.5	Test Procedures
3.6	Ethical and Safety Implementation Notes
3.7	Additional Visual Analysis
Chapter 4	DETAILED TECHNICAL DOCUMENTATION

4.1	Additional Visual Analysis	34
4.2	Fraud Dependency Graph Technical Notes	35
4.3	Conditional WGAN-GP and FiLM Details	36
4.4	Conditional Aware Attention Details	36
4.5	Tactical Deployment Agent Details	37
4.6	Tactical Deployment Agent Details	38
4.7	Result/Interpretation in Academic Terms	39
4.8	Hardware Efficiency Discussion	39
4.9	Ethical Safeguard	40
4.10	Detailed Chapter Summary	40
Chapter 5	CONCLUSION	41
5.1	Final Scope of Project	41
5.2	Drawbacks and Limitations	42
REFERENCES		44
ANNEXURES		46

1. INTRODUCTION

1.1 *Project Overview*

Deceptive-Net is a cyber deception and honey-token generation framework. According to the source paper, honey-tokens are convincing but useless items that when exfiltrated or misused, indicate malicious activity. The project centers on solving the believability problem: Static hand-crafted fake records can be fingerprinted by highly trained attackers, and general-purpose tabular synthesis methods do not enforce semantic or causal constraints that are required for deceptive identities.

The proposed framework incorporates causal modelling, generative modelling, neuro-symbolic constraint checking, reinforcement learning, and honeypot deployment. It has the following major components: Fraud Dependency Graph, Conditional WGAN-GP generator with FiLM conditioning, multi-head critic, Credential-Aware Attention module, and the DQN Tactical Deployment Agent. They are offered as a coherent architecture for honey-tokens creation and distribution for Cowrie SSH/Telnet honeypots.

Deceptive-Net is a Cyber-deception and a Honey-token generation framework. According to the source paper, the honey-tokens are convincing but harmless entities that breach or misuse indicate malicious activity. The project is concerned with the believability problem, whereby, experienced attackers can be able to fingerprint static hand-crafted fake records, yet general purpose tabular synthesis methods are not directly leveraging all semantic and causal constraints for deceptive identities.

The proposed framework is a hybrid of causal modelling, generative modelling, neuro-symbolic constraint checking, reinforcement learning, and deployment of honeypots. The main modules are the Fraud Dependency Graph, Conditional WGAN-GP generator with FiLM conditioning, multi-head critic, Credential-Aware Attention module and the DQN Tactical Deployment Agent. They are described in a single architecture for the creation and deployment of honey-tokens in Cowrie SSH/Telnet honeypots.

1.2 *Existing System and Need for System*

The three limitations of the existing systems identified in the source paper are that: The three limitations identified in the source paper are: Static honey-token systems use a fixed bait which is not adapted to the attacker's behaviour. While these general purpose tabular generative models (CTGAN, TVAE, GReaT, TabDDPM, CopulaGAN, Gaussian Copula, SMOTE-NC) are able to generate tabular records, they lack consideration of fraud causality and do not impose semantic credential constraints. While existing honeypot solutions like Cowrie offer interactive deceptions, most bait credentials are handmade.

The challenge of Deceptive-Net comes from the demand to create tokens that are distributionally realistic and semantically coherent, as well as placed adaptively. The source design accounts for this by introducing synthetic PII fields and integrating reinforcement learning to counterattacker command entropy, dwell time, and detection events -- all in the pursuit of exfiltration.

1.3 Scope of Work

- Generate behaviourally plausible fake digital identities, financial records, and session credentials.
- Model fraud-aware causal dependencies using a Fraud Dependency Graph over BAF features.
- Train a Conditional WGAN-GP generator with FiLM conditioning and a multi-head critic.
- Apply Credential-Aware Attention to preserve Luhn, geo-semantic, semantic, and temporal constraints.
- Use a DQN Tactical Deployment Agent for attacker-adaptive token placement in Cowrie honeypots.
- Evaluate the system using DDS, DRS, Luhn validity, name-email coherence, throughput, ablation studies, and deployment metrics.

1.4 Operating Environment

Table 1.1 Operating Environment

Component	Specification from Source
GPU	NVIDIA GTX 1650 Ti, 4 GB GDDR6
Host	Intel Core i7 host
RAM	16 GB
Honeypot	Cowrie SSH/Telnet honeypot
Training optimisation	AMP FP16, gradient checkpointing, sparse GAT message passing, LoRA rank 8
Reported VRAM	3.5 GB training, 2.1 GB inference

1.5 Related Work Summary

The source paper classifies Deceptive-Net++ into five categories of previous research: honey-token and honeypot systems, tabular generative models, conditional GANs, causal generative models, and reinforcement learning for security. A honey token system is defined as “digital objects whose value is in their ability to identify misuse”. Cowrie is said to be an easy-to-access SSH/Telnet honeypot platform; however, the source claims that bait credentials are not usually created by a high fidelity adaptive model, but are instead made by hand.

CTGAN and TVAE are referred to as tabular generation methods based on GANs. TabDDPM is a denoising diffusion model for tabular data, and GReaT serialises tabular rows and generates using the language-model approach. Baselines are also mentioned: CopulaGAN, Gaussian Copula and SMOTE-NC. The stated issue is that these approaches produce records, but they don't explicitly model the causation of fraud, the validity of semantically valid credentials or the cyber-deception use case.

The generative backbone is given by Conditional GANs and WGAN-GP. The source uses WGAN-GP to train in a stable manner and FiLM to condition features-wise. This is expanded by the project's multi-head critic and Credential-Aware Attention. CausalGAN encourages the application of causal structure, while Deceptive-Net++ leverages causal graph conditioning on tabular features of fraud, instead of image features.

The inclusion of reinforcement learning is due to the fact that the deployment of honey token is considered as a sequential decision problem. As far as the source paper is concerned, the deployment scheduling uses DQN, and the entropy of the command sequence of attackers is part of the observed state. This enables the system to decide if a financial, credential, identity, or none of the extra token should be deployed based on Cowrie observations.

1.6 Threat Model

The source threat model is for technologically capable automated, bots, semi-automated, and moderately-skilled human operators that follow scripted playbooks. It makes no such promise to hoodwink nation-state manual operators with unlimited time. This boundary becomes significant because this project has considered the plausibility in the light of realistic but not omniscient attackers.

The attacker is assumed to have SSH/Telnet access to an internet-facing honeypot machine, which will then allow the attacker to execute shell commands, scan for files, steal credentials, and possibly run basic Luhn checks against card numbers. The attacker is unable to access the honey-token generator and its training data. The defender's goal is to maximize dwell time, exfiltration-attempts and token reuse attempts and minimize the number of detected-synthetic tokens to below 5%.

There are also a number of security assumptions that are made in the source. The generator is isolated from production using air-gaps. The honeypot file system is protected by a secure orchestration channel to inject generated tokens. The DQN agent only reads Cowrie JSON logs and has no access to production infrastructure. These assumptions are used to establish the safe operating envelope of the system.

2. BACKGROUND MATERIAL

2.1 *Honey-Token and Honey-pot Systems*

The initial idea of using a honeypot as a digital intrusion detection object was described in 2003 by Lance Spitzner in his article, *Honeytokens: The Other Honey-pot*. Honey-tokens are any type of digital entity, such as username, password, credit card information, document, API key, etc., that are solely used to prevent access. Honey Tokens are any digital object, including a username, password, credit card number, document or API key, used to prevent access. Honey-tokens are not honeypots, which can simulate an entire computing system, but rather light-weight and can be integrated in the production systems without being obvious, which makes them very scalable.

This idea has been operationalised by Yuill et al. (2004) who used the term "Honeyfiles," which are files strategically placed in file systems to trap intruders by monitoring access. Their work showed they could successfully signal malicious activity even with basic static honey-tokens, if they were well instrumented. But the lack of dynamism in the tokens was already evident – if an attacker could learn how fake records are named or how they are structured, they lose the value of the tokens.

Cowrie, written and maintained by Michel Oosterhof, is the most important open source project to operational honeypot research. Cowrie acts as a SSH/Telnet server and captures all attacker commands, session sequences, uploaded/downloaded files and timing information in structured JSON logs. Cowrie's telemetry provides the richness and is thus the de facto standard for honey-token interaction research. Deceptive-Net++ is integrated into Cowrie's logging infrastructure, and receives a stream of log events in JSON format as the state signal for the DQN Tactical Deployment Agent.

2.2 *Tabular Generative Models*

The challenge of generating realistic synthetic tabular data, i.e., data stored in a tabular format with an array of mixed numerical, categorical, ordinal types, has been the topic of much research after the success of generative adversarial networks in image generation. The works of Xu et al. (2019) in NeurIPS 2019, CTGAN and TVAE, were introducing the GAN-based tabular synthesis, which tackles two fundamental challenges for tabular data: multimodal distributions of continuous columns (using mode specific normalisation) and imbalanced categorical columns (using conditional sampling vectors). In the financial record generation domain, these are the primary comparison baseline methods used for Deceptive-Net.

TabDDPM was proposed by Kotelnikov et al. at ICML 2023, and tackles denoising diffusion probabilistic models for tabular data, outperforming GAN based approaches in terms of statistical fidelity on several standard benchmarks. This new method, GReaT, introduced by Borisov et al. at ICLR 2023, works quite differently by fine-tuning a state-of-the-art language model (in this case, GPT-2) to generate a large number of rows from a tabular data collection, using the language model's own world knowledge to create the data. CopulaGAN and Gaussian Copula are two models where inter-column dependencies are modeled using copula functions that are statistical functions that partition the marginal distributions from the dependencies. SMOTE-NC is an oversampling algorithm that is an extension of the popular

SMOTE algorithm for handling mixed-type data. None of these approaches address fraud-propagation causal structure or take into account semantic credential constraints.

2.3 *Conditional GANs and WGAN-GP*

The idea of Conditional Generative Adversarial Networks (cGANs), proposed by Mirza and Osindero in 2014, is to add an additional auxiliary input to the standard GAN, usually a class label or a feature vector. For Deceptive-Net++ the condition formulation is important to provide directed generation of samples that meet certain properties – these properties are required to be in relation to the fraud risk tier and causal graph embeddings.

The Wasserstein GAN with Gradient Penalty (WGAN-GP), proposed by Gulrajani et al. at NeurIPS 2017, overcomes the issues with training instability faced by the standard GANs, using the Wasserstein-1 (Earth Mover's) distance objective instead of the Jensen-Shannon divergence objective, and imposing the Lipschitz constraint by adding a gradient penalty term to the loss function (as opposed to clipping weights). WGAN-GP is empirically more stable, offers a useful training loss as a convergence measure and does not suffer from mode collapse, which can be important when generating tabular data where some of the columns may have heavy-tailed or multimodal distributions.

The conditioning mechanism introduced by Perez et al. (AAAI 2018), called Feature-wise Linear Modulation, learns affine transformations (scale gamma and shift beta) element-wise on intermediate feature representations, offering a powerful and light conditioning mechanism. In Deceptive-Net++ the generator can adjust its internal representations based on the fraud dependency context through a process called “FiLM conditioning,” with minimal computational burden. This is essential for the fraud-aware generation needed by the FiLM layers, as they are able to create different activation patterns for fraudulent and legitimate identity profiles.

2.4 *Casual Generative Models*

CausalGAN, proposed by Kocaoglu et al. at ICLR 2018, was the first work to incorporate causal graphs into the GAN framework, demonstrating that causal conditioning enables intervention and counterfactual generation that purely statistical models cannot perform. CausalGAN operated in the image domain using a causal graph over semantic attributes (e.g., "wearing glasses causes a certain hairline shape"). Deceptive-Net++ extends this causal generative modelling paradigm to the tabular domain through the Fraud Dependency Graph.

The Fraud Dependency Graph (FDG) models conditional dependencies among BAF fraud features — specifically, the causal pathway: $\text{foreign_request} \rightarrow \text{velocity_}^* \rightarrow \text{device_fraud_count} \rightarrow \text{fraud_bool}$. This causal structure is exploited by a Graph Attention Network (GAT) to produce a compact causal embedding vector that conditions the CWGAN-GP generator. The key distinction from CausalGAN is the domain: tabular financial and PII data, with domain-specific causal priors derived from fraud propagation literature rather than visual semantics.

2.5 *Reinforcement Learning for Security*

Up to now, no study has introduced causal graphs into the GAN framework, and Kocaoglu et al. (2018) were the first to show that causal conditioning is capable of intervention and counterfactual generation which can only be achieved by purely statistical models. CausalGAN was implemented in the image space, where it was based on a causal graph over semantic attributes (e.g., "wearing glasses implies a specific hairline shape"). Deceptive-Net++ further generalizes the causal generative modelling paradigm for the tabular space via the Fraud Dependency Graph.

A Fraud Dependency Graph (FDG) models conditional dependencies between the BAF fraud features, specifically the causal pathway: `foreign_request -> velocity_* -> device_fraud_count -> fraud_bool`. A Graph Attention Network (GAT) is used to generate a compact causal embedding vector which conditions the CWGAN-GP generator, which exploits this causal structure. The difference over CausalGAN is the domain: tabular financial and PII data, instead of visual semantics, where domain-specific causal priors are taken from fraud propagation literature.

In recent years, there have been some cyber security areas where reinforcement learning (RL) has been applied. The network intrusion detection problem was tackled with deep RL by Sethi et al., which they formulated as a sequential decision process over network traffic streams. Bhattacharya et al. proposed a new framework in automated penetration testing called AutoPentesting using deep RL to find the vulnerability chains in network topologies. Both works provide an evidence for the possibility of using RL for decision making in cybersecurity in uncertain environments. The use of DQN for deployment scheduling of honey tokens is novel in the literature.

The idea is that the type of honey-token (financial documents, credentials, or identity documents) to be deployed, and the time to refresh the corpus of the honey-tokens, can be defined as a partially observable Markov Decision Process with the attacker's command-sequence entropy in Cowrie logs as an informative state signal. The formulation enables the deployment agent to learn attacker-preference-aware strategies which maximize the amount of time they can spend and the number of exfiltrations they conduct without any synthetic-token detection.

2.6 Technologies Involved

This section summarises the key technologies employed in the Deceptive-Net++ framework.

Technology	Category	Role in Deceptive-Net++
CWGAN-GP	Deep Generative Model	Core honey-token generation
Graph Attention Network (GAT)	Graph Neural Network	FDG causal embedding
FiLM Conditioning	Conditional Modulation	Fraud-aware generation
Credential-Aware Attention	Neuro-Symbolic Module	Constraint enforcement
Deep Q-Network (DQN)	Reinforcement Learning	Adaptive token deployment
Cowrie Honeypot	Security Infrastructure	Live deployment & telemetry
BAF Dataset (NeurIPS 2022)	Training Data	Financial fraud features
Mendeley Synthetic PII	Training Data	PII credential features
Luhn Algorithm	Symbolic Constraint	Credit-card validity check
Sentence-Transformers	NLP	Email-name semantic coherence
Automatic Mixed Precision	Hardware Optimisation	FP16 GPU efficiency
LoRA (r=8)	Parameter Efficiency	Decoder weight compression

Table 2.1: Key Technologies in Deceptive-Net++

3. SYSTEM ANALYSIS AND DESIGN

3.1 Project Overview

The project is technically feasible according to the source paper because the full model contains 4.9 million trainable parameters and fits within a 4 GB consumer GPU memory budget. The source reports 3.5 GB peak training VRAM and 2.1 GB peak inference VRAM with FP16 automatic mixed precision

Table 3.1 Operating Environment

Aspect	Observation
Technical feasibility	Uses WGAN-GP, FiLM, GAT, CAA, DQN, and Cowrie; source reports consumer GPU execution.
Operational feasibility	Tokens are deployed in a Cowrie honeypot environment and observed through command logs.
Performance feasibility	Reported throughput is 10.0 k tokens/s in profiling and 14.5 k tokens/s in comparison results.
Ethical feasibility	Source limits deployment to owned infrastructure and does not integrate real financial infrastructure.

3.2 Software and Hardware Requirement Specifications

Table 3.2 Software and Hardware Requirement Specifications

Type	Requirement
Software	Cowrie SSH/Telnet honeypot, training code, evaluation metric implementations, DQN environment.
Model algorithms	Conditional WGAN-GP, FiLM conditioning, GAT, multi-head critic, CAA, DQN.
Datasets	NeurIPS 2022 BAF benchmark and Mendeley Synthetic PII corpus.
Hardware	NVIDIA GTX 1650 Ti 4 GB GPU, Intel Core i7 host, 16 GB RAM.

3.3 Data Flow Diagram (DFD)

The context-level DFD shows the researcher or SOC operator configuring the Deceptive-Net++ framework. BAF and PII datasets provide training material. The framework generates and deploys honey-tokens into Cowrie, and Cowrie logs return attacker-state observations.

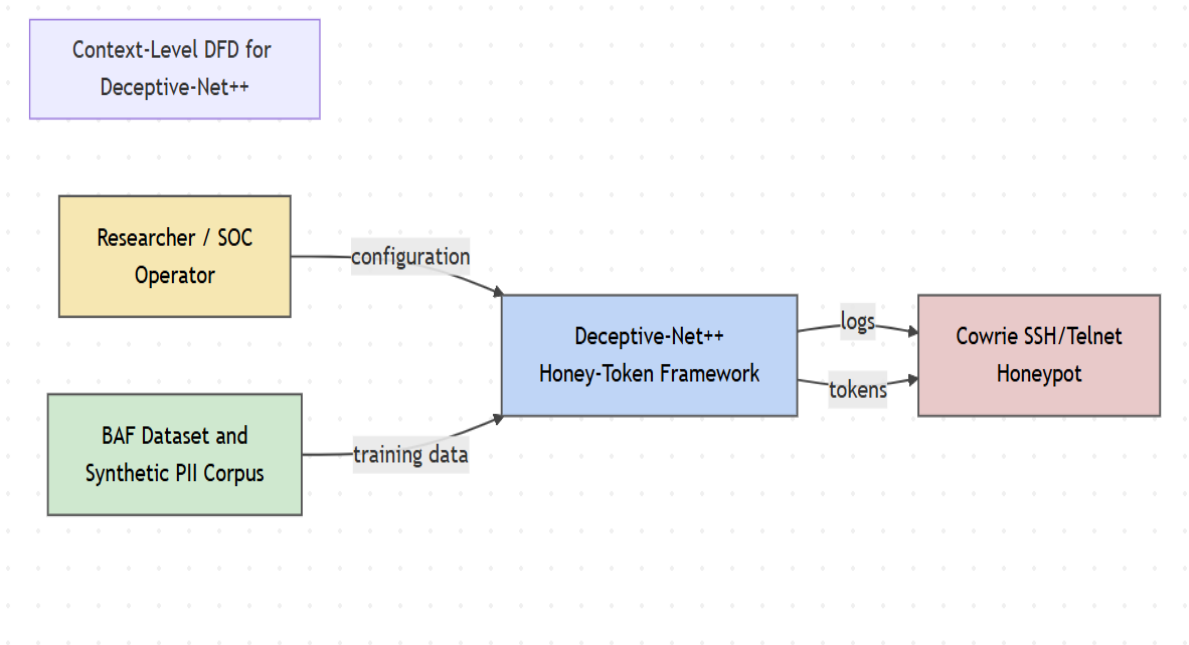


Figure 3.1 Context-Level DFD for Deceptive-Net

The level-1 DFD expands the internal flow. BAF and PII records feed the Fraud Dependency Graph and generator. The CAA module applies symbolic constraints. The token store provides candidate honey-tokens to the DQN Tactical Deployment Agent, which interacts with Cowrie logs through state and reward feedback.

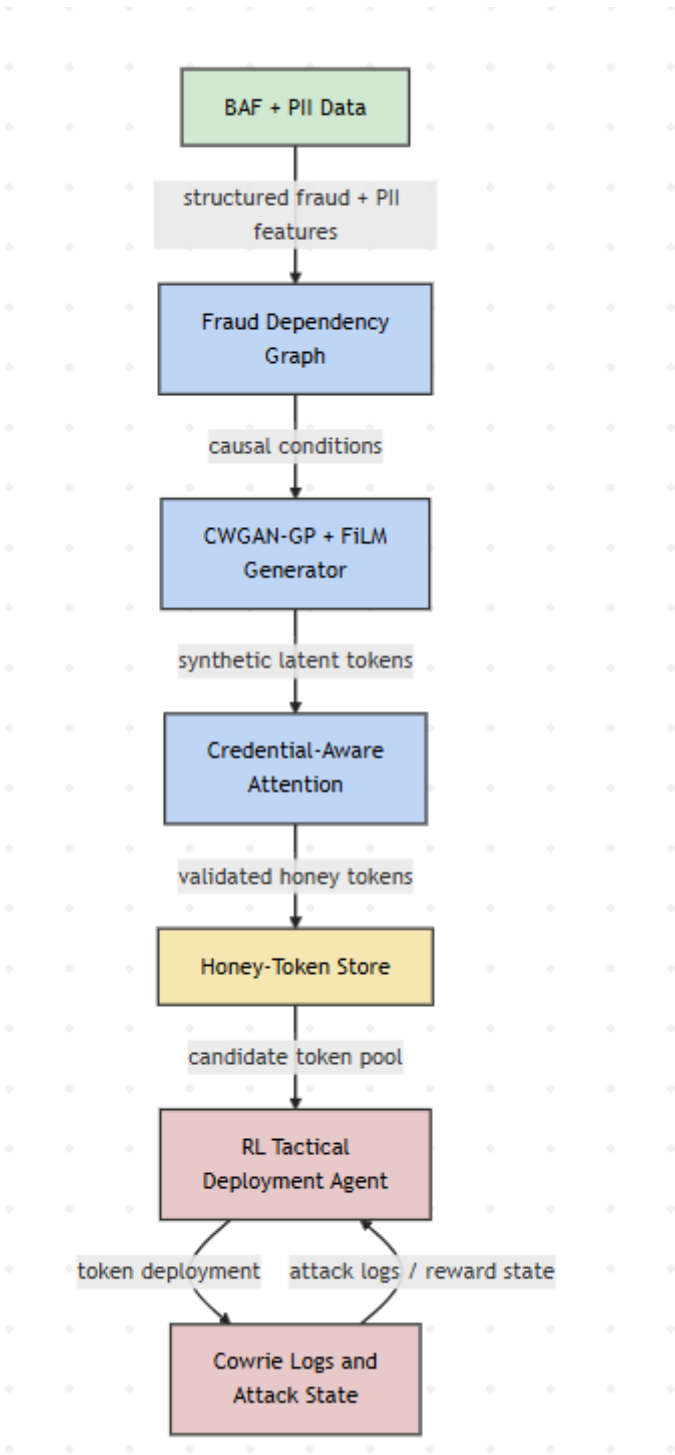


Figure 3.2 Level-1 Data Flow Diagram

3.4 Functional Decomposition Diagram (FDD)

The functional decomposition separates the framework into causal feature modelling, honey-token generation, constraint enforcement, adaptive deployment, and evaluation/profiling. This decomposition follows the five-module system described in the source paper.

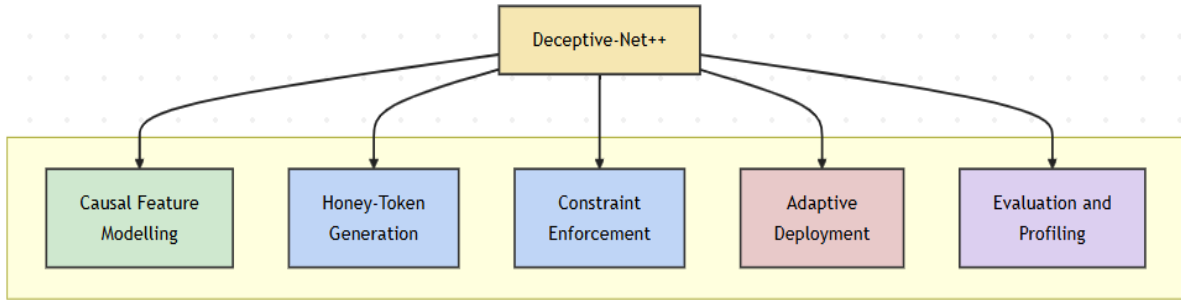


Figure 3.3 Functional Decomposition Diagram

3.5 Entity Relationship Diagram (ERD)

The source describes two dataset domains: BAF records and Mendeley synthetic PII records. A record-alignment step pairs synthetic PII records with BAF samples based on fraudulent risk tier, preserving cross-domain semantic coherence. Generated honey-tokens are then selected for deployment, and Cowrie logs provide observed events.

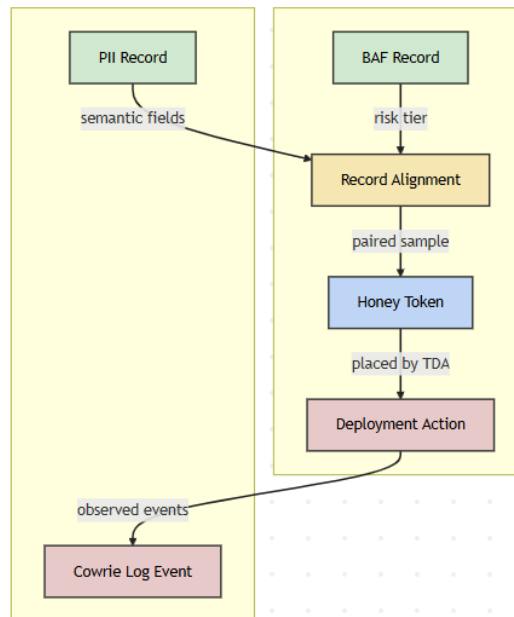


Figure 3.4 Entity Relationship View

3.6 Data Dictionary and Table Design

Table 3.3 BAF Dataset Feature Group Summary

Group	Feature	Representative Feature(s)
Target Variable	1	fraud_bool
Applicant Demographics	4	income, customer_age
Application Details	6	proposed_credit_limit
Identity / Contact Verification	4	name_email_similarity
Historical / Address Data	5	bank_months_count
Velocity / Frequency	6	velocity_6h, velocity_24h
Session / Device Info	6	device_fraud_count

Table 3.4 Mendeley Synthetic PII Domain Summary

Domain	Requirement
Personal Identity	Name, DOB, Education Level, Government-Issued ID type
Contact and Location	US-formatted phone, e-mail, street address, city, state, ZIP
Financial Data	16-digit Luhn-compliant card number, CVV, expiry
Digital Credential	Usernames, passwords, OS/device descriptor string

Table 3.5 Data Dictionary

Data_Item	Meaning / Use
fraud_bool	Binary target in the BAF dataset.
name_email_similarity	Representative identity/contact verification feature.
velocity_6h and velocity_24h	Velocity/frequency features used in fraud dependency modelling.
device_fraud_count	Session/device metric in the BAF feature groups.
Luhn Validity	Constraint-related measure for generated credit-card numbers.
Command entropy	Attacker-state element observed from Cowrie command sequences.

3.7 Detailed Architecture Design

The Fraud Dependency Graph is a directed weighted graph of BAF features. An edge represents a causal relationship between two features in fraudulent transactions, and the weight of the edge is proportional to the conditional mutual information between the features. In the source, the main path identified as the causal path is `foreign_request`, `velocity` features, `device_fraud_count`, `fraud_bool`. This graph can be used as a causal context to the generator rather than assuming that every feature is independent.

The Fraud Dependency Graph is used to generate a graph's adjacency matrix, which is then used to train a Graph Attention Network. The output is a 128 dimensional causal embedding h_c . This embedding is combined with the fraud condition and inserted with the fraud condition into the condition vector that is passed to the generator. The design thus leverages graph-derived fraud context as one controlling signal for the generation of synthetic honey-tokens.

Given a latent noise vector z and a condition c , the generator is used to generate a synthetic honey-token. Hidden activations are modulated by learned gamma and beta projections in each residual block, which is known as FiLM conditioning. FiLM is used due to its light weight feature level conditioning without high level of multiplicative overhead. The source points name FiLM as one of the most crucial ones in the ablation outcome.

The critic is broken down into BAF and PII heads. The combined critic score is the weighted sum of the BAF realism head and the PII realism head ($\alpha=0.6$ in the source). This is a multi-head design so the discriminator cannot give more importance to one domain over another. This is important in the project context because in order to make the honey-tokens real, they need to be realistic as a financial/fraud record and as an identity/credential record..

WGAN-GP is based on the Wasserstein distance with gradient penalty. The source employs the values of $\lambda_{gp} = 10$ and five critic steps per generator step. The gradient penalty helps to maintain stability in training by limiting the norm of the critic gradient and the multiple critic steps further ensure a more consistent update of the generator.

Credential-Aware Attention sits in between the last generator layer and output decoder. It employs a method of attention gated projection onto a feasible constraint set. We have a set of four constraint losses, which are all stated: Luhn validity, geographic consistency, semantic name-email coherence, and temporal velocity ordering. The total generator loss is a composition of WGAN loss and constraint-preservation loss.

The Tactical Deployment Agent models the problem of placing honey-tokens as a finite-horizon Markov Decision Process. The state contains the last 10 Cowrie commands encoded as TF-IDF vectors, an entropy of the command sequence, a number of exfiltrations and dwell time. The deploy financial, deploy credentials, and deploy identity actions are in the action space, along with the hold action. Reward is a combination of dwell-time improvement, exfiltration signal and a negative reward for encountering a synthetic token.

In the source, the implementation of the DQN asked a dueling structure and emphasized on prioritised experience replay. Replay buffer size is set to 10^5 , batch size to 256, and network updates to the target are made every 10^3 steps. The online standardisation of the observations is performed by using running mean and variance.

3.8 Evaluation Metric Design

The Deception Discriminability Score is a score of the difficulty with which a downstream classifier can separate synthetic honey-tokens from real records. The definition of DDS is created with a random forest classifier that is trained to classify between synthetic and real samples, and the out-of-sample brier score is used. The lower the DDS the better, because they are lower discriminability.

The Deception Realism Score is said to be a comprehensive real score. It includes inverse DDS, semantic coherence, Luhn validity and normalised attacker engagement. The source reports the following weights: $\alpha=0.4$, $\beta=0.25$, $\gamma=0.2$, and $\delta=0.15$, based on a red team evaluation of 50 batches of honey.

These are valuable metrics due to the fact that standard machine-learning metrics are not enough for cyber deception. A token that appears to be statistically plausible but does not pass the Luhn test or the name-email coherence test can be identified rapidly. Likewise, if a token has a strong distribution but little effect on dwell time then it may not be of any help operationally. Therefore, DDS and DRS are advocated as deception-oriented measures.

3.9 Code Design

The source paper does not provide a full source-code listing. Therefore, this section presents implementation-oriented pseudocode derived from the described algorithms and equations.

Pseudo-code: train FDG-conditioned CWGAN-GP.

Step 1: load BAF and PII records.

Step 2: construct the Fraud Dependency Graph using conditional mutual information.

Step 3: train a Graph Attention Network to produce causal embedding hc .

Step 4: sample latent vector z and condition $c=[hc; y_{\text{fraud}}]$.

Step 5: generate a candidate honey-token with the FiLM-conditioned generator.

Step 6: evaluate BAF and PII realism using the multi-head critic.

Step 7: apply CAA constraint loss.

Step 8: optimise WGAN-GP objective with gradient penalty.

Code: **main.py**

```
# =====  
# Deceptive-Net v1.0 Research Edition  
# Main Execution Controller  
# =====  
  
import os  
import argparse  
import logging  
from datetime import datetime  
# =====
```

```

# Import Project Modules
# =====
from utils.data_loader import load_dataset
from utils.preprocessing import preprocess_data
from utils.model import train_model
from utils.evaluation import evaluate_model
from utils.generator import generate_honey_tokens
from utils.visualization import generate_graphs
from utils.report_generator import generate_reports
from utils.runtime_manager import RuntimeManager
from utils.threat_detection import ThreatDetectionSystem
from utils.network_monitor import NetworkMonitor
from utils.honeypot import HoneypotManager
from ui.dashboard import launch_dashboard
# =====
# Logging Configuration
# =====
LOG_DIR = "logs"
if not os.path.exists(LOG_DIR):
    os.makedirs(LOG_DIR)
logging.basicConfig(
    filename=os.path.join(LOG_DIR, "execution.log"),
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)
logger = logging.getLogger("Deceptive-Net")
# =====
# Banner Function
# =====
def show_banner():
    print("=" * 60)
    print("  DECEPTIVE-NET CYBERSECURITY PLATFORM")
    print("=" * 60)
# =====
# Main Function
# =====
def main():

```

```

show_banner()

# =====
# Argument Parser
# =====

parser = argparse.ArgumentParser(
    description="AI-Powered Cybersecurity and Deception System"
)

parser.add_argument(
    "--mode",
    type=str,
    default="train",
    choices=[
        "train",
        "evaluate",
        "generate",
        "dashboard",
        "monitor",
        "honeypot",
        "full"
    ],
    help="Execution Mode"
)

parser.add_argument(
    "--dataset",
    type=str,
    default="data/Base.csv",
    help="Path to dataset"
)

parser.add_argument(
    "--epochs",
    type=int,
    default=50,
    help="Number of training epochs"
)

parser.add_argument(
    "--device",
    type=str,

```

```

        default="cpu",
        help="Execution device (cpu/cuda)"
    )
args = parser.parse_args()
logger.info("=====")
logger.info("Deceptive-Net Execution Started")
logger.info(f"Mode: {args.mode}")
logger.info(f"Dataset: {args.dataset}")
logger.info(f"Device: {args.device}")
logger.info("=====")
# =====
# Dataset Loading
# =====
logger.info("Loading dataset...")
dataset = load_dataset(args.dataset)
logger.info("Preprocessing dataset...")
processed_data = preprocess_data(dataset)
# =====
# Train Mode
# =====
if args.mode == "train":
    logger.info("Starting model training...")
    train_model(
        data=processed_data,
        epochs=args.epochs,
        device=args.device
    )
    logger.info("Training completed successfully.")
# =====
# Evaluation Mode
# =====
elif args.mode == "evaluate":
    logger.info("Running evaluation pipeline...")
    evaluate_model(processed_data)
    logger.info("Evaluation completed.")
# =====
# Honey Token Generation

```

```

# =====
elif args.mode == "generate":
    logger.info("Generating synthetic honey tokens...")
    generate_honey_tokens()
    logger.info("Honey token generation completed.")
# =====
# Real-Time Monitoring
# =====
elif args.mode == "monitor":
    logger.info("Starting network monitor...")
    monitor = NetworkMonitor()
    monitor.start_monitoring()
# =====
# Honeypot Services
# =====
elif args.mode == "honeypot":
    logger.info("Launching honeypot services...")
    honeypot = HoneypotManager()
    honeypot.start_services()
# =====
# Dashboard Mode
# =====
elif args.mode == "dashboard":
    logger.info("Launching dashboard system...")
    runtime = RuntimeManager()
    runtime.start()
    launch_dashboard()
# =====
# Full Pipeline Execution
# =====
elif args.mode == "full":
    logger.info("Running complete pipeline...")
    # Step 1 - Train Model
    train_model(
        data=processed_data,
        epochs=args.epochs,
        device=args.device

```

```

)
# Step 2 - Evaluate Model
evaluate_model(processed_data)
# Step 3 - Generate Honey Tokens
generate_honey_tokens()
# Step 4 - Threat Detection
detector = ThreatDetectionSystem()
detector.run_detection(processed_data)
# Step 5 - Generate Graphs
generate_graphs()
# Step 6 - Generate Reports
generate_reports()
# Step 7 - Launch Dashboard
runtime = RuntimeManager()
runtime.start()
launch_dashboard()
logger.info("Complete pipeline executed successfully.")
# =====
# Finish Execution
# =====
logger.info("=====")
logger.info("Execution Finished Successfully")
logger.info(f"Time: {datetime.now()}")
logger.info("=====")
print("\n[+] Deceptive-Net execution completed.\n")
# =====
# Entry Point
# =====
if __name__ == "__main__":
    main()

```

Pseudo-code: DQN Tactical Deployment Agent.

Step 1: observe the last ten Cowrie commands, command entropy, exfiltration count, and dwell time.

Step 2: choose one action from deploy financial, deploy credentials, deploy identity, or hold.

Step 3: compute reward from dwell-time improvement, exfiltration signal, and detected-synthetic penalty.

Step 4: update the dueling DQN using prioritised replay.

Step 5: refresh target network periodically.

3.10 Menu Tree and Screen Design Consideration

This is a research paper and is not a typical menu-driven application. From a report-design perspective, the system can be summarized as a set of 4 analytical views based on a source workflow: dataset analysis, feature analysis, honey-token generation, deployment monitoring and evaluation/profiling. These views are likely to correspond to the main functions described in the framework, not necessarily to a particular menu option that is presented to the user.

Input screens are also not listed in the source as graphical forms. The inputs that are effective include: training hyperparameters, command logs from Cowrie, fraud labels, Mendeley PII records, and BAF records. Outputs are generated honey-tokens, DDS/DRS metrics, Luhn and semantic coherence scores, deployment actions, dwell-time statistics, and profiling measurements.

4. TEST PROCEDURES AND IMPLEMENTATION

4.1 *Project Code and Algorithm Snippets*

The five technical aspects that are at the heart of the project implementation outlined in the source are the following: FDG, CWGAN-GP with FiLM, multi-head critic, CAA, and DQN TDA. The generator is a model that generates a synthetic honey-token from a latent vector and causal/fraud condition. In FiLM conditioning, learned projections are applied to internal activations. The critic uses a weighted combination to decompose realism into BAF and PII heads.

The CAA module operates between the last generator layer and output decoder. It includes Luhn validity, geo-semantic consistency, name-email semantic coherence, and temporal velocity ordering into a constraint loss. The deployment of the DQN TDA models is modeled as a finite-horizon MDP that depends on the Cowrie command observations, entropy, exfiltration attempts, and dwell time.

4.1.1 *Training Objective*

The training goal is formulated according to the WGAN-GP formulation. The difference between real and generated samples is maximised by the critic, and the Lipschitz penalty is imposed by the gradient. The generator reduces the loss from the constraint-preservation loss and the adversarial component. The source reports that the number of critic steps used for each generator step is 5 and λ_{gp} is 10.

The constraint-preservation loss is made up of four terms. Application of Luhn loss forces generated credit-card numbers to conform to check-digit structure. Geographic loss: KL-divergence style formulation – City conditioned on ZIP. Semantic loss is based on the cosine similarity of embedded e-mail and name. Temporal loss is an ReLU loss on `velocity_6h` and `velocity_24h` that punishes invalid velocity ordering.

4.1.2 *Deployment Agent Procedure*

The DQN agent gets the state from Cowrie and chooses the token placement actions. The pos/reward provided in the source from dwell-time and exfiltration signals is positive contribution, while tokens are detected the contribution is negative. This is a connection between the defender's goal of increasing the engagement of the attacker and the reduction of synthetic-token recognition that is direct in this design.

In the source, the action distribution approaches the policy of "Financial tokens first. This is explained by the learning curve of the DQN, namely that the cumulative reward increases after 500 training sessions, and that the estimates of the Q values converge after 200 episodes.

4.2 Screenshots and Project Figures

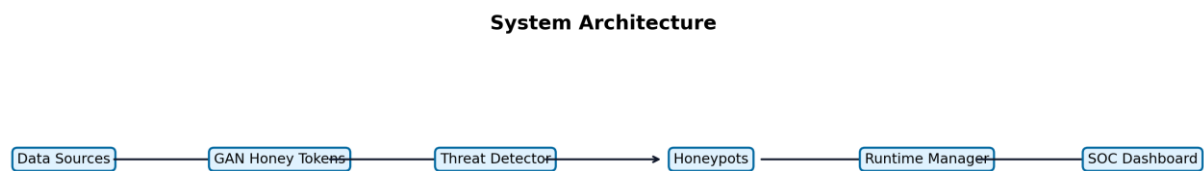


Figure 4.1 Deceptive-Net++ System Architecture

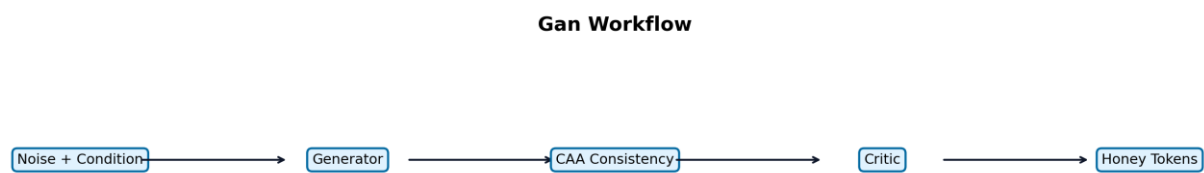


Figure 4.2 Training Dynamics Figure

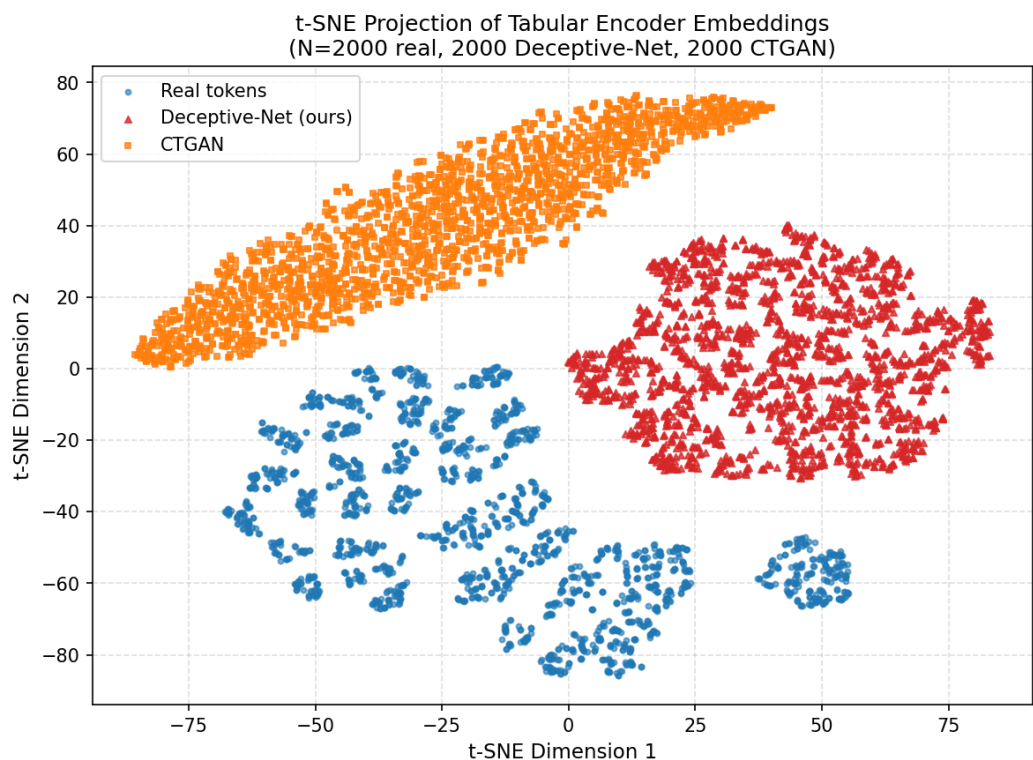


Figure 4.3 t-SNE Distributional Comparison

4.3 Experimental Setup and Results

Table 4.1 Experimental Setup

Parameter	Value from Source
BAF split	800k training records, 100k validation records, held-out NeurIPS test partition
Mendeley PII	All 100k records used jointly
Generator	6 residual blocks, dh=256, GELU, spectral normalisation
Critic	4-layer MLP per head, LeakyReLU, spectral normalisation
CAA	4-head cross-attention, datt=128
Optimizer	Adam, beta1=0.5, beta2=0.9
Learning rates	LRG=2e-4, LRD=1e-4
Training	400 epochs, batch size 512

Table 4.2 Main Comparison Results

Method	DDS ↓	Luhn% ↑	Name-Email% ↑	k tok/s ↑
Static Tokens	109.274	10.9	75.5	-
TabDDPM	32.855	26.3	67.1	136.2
GReaT	31.509	12.5	75.7	160.7
TVAE	31.316	15.4	65.6	138.2
CopulaGAN	28.824	14.4	69.1	238.9
CTGAN	16.964	17.2	72.0	269.6
Gaussian Copula	1.819	10.3	79.5	41.6
SMOTE-NC	1.423	7.7	78.7	848.7
Deceptive-Net++	0.669	35.9	90.7	14.5

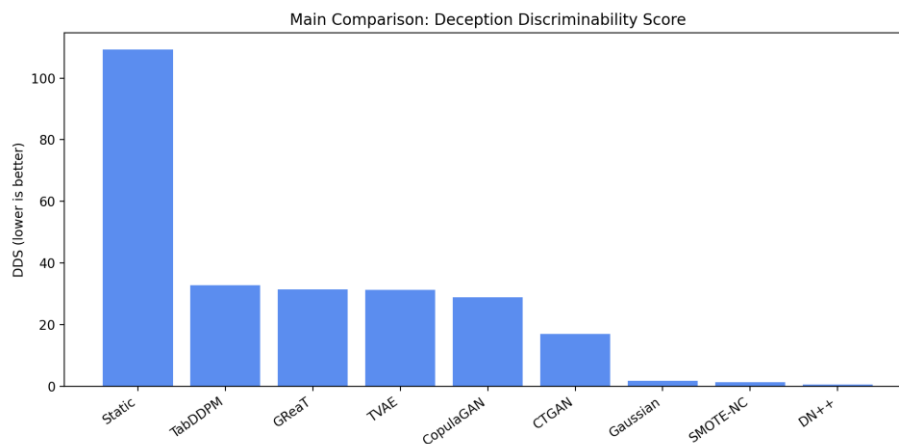


Figure 4.4 DDS Comparison Plot

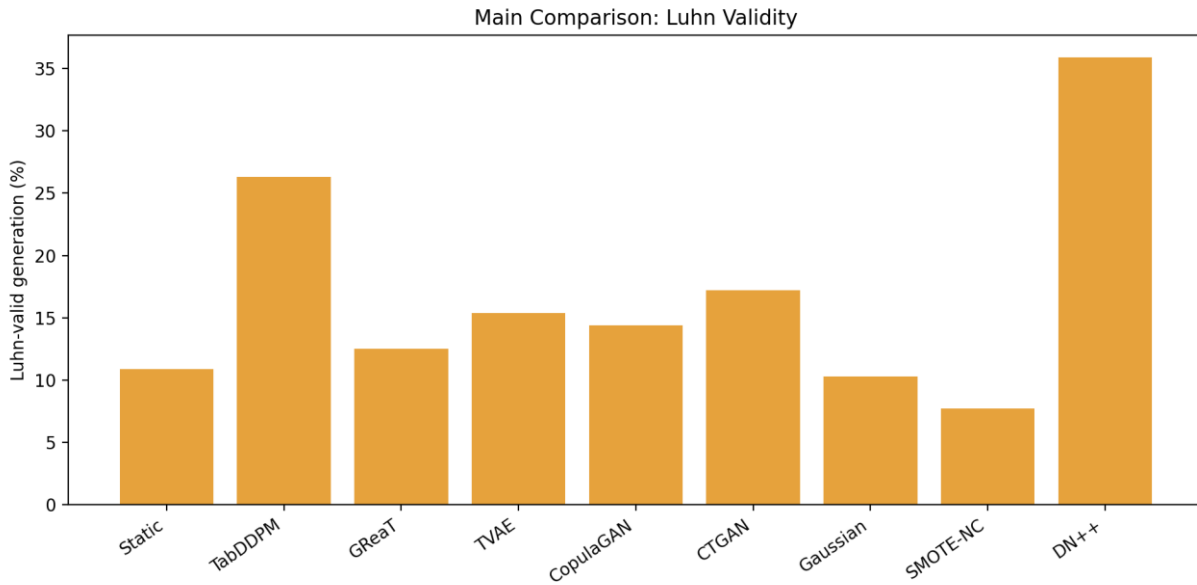


Figure 4.6 Luhn Validity Plot

According to the source, the Deceptive-Net++ has a DDS of 0.669 while SMOTE-NC has a DDS of 1.423. It also states that 35.9% of cards generated are Luhn valid and that 90.7% of names and emails are semantically coherent. It suffers from lower throughput than a few baselines due to the inclusion of overhead of FDG graph inference, but still is suitable for real-time deployment as per the source.

It is also noteworthy from the table that Gaussian Copula and SMOTE-NC have low DDS values among most of the neural baselines, but neither method is explicitly described to enforce a mixture of fraud causality, credential constraints, and deployment adaptation. Deceptive-Net++ is thus considered not only as a generator but also as a deception framework having distributional and semantic goals as well as engagement goals.

The source emphasises the name-email coherence value of 90.7% which is more than 11.2 percentage points higher than any indicated baseline. Attackers could check if the usernames are congruent with the e-mail address and identities. This coherence is furthered by CAA and by the record alignment across domains in the source design.

Table 4.3 Ablation Study Results

Config	Description	DDS ↓	Luhn% ↑	ΔDDS
A	Full Deceptive-Net++	0.669	35.9	-
B	A minus CAA Module	128.861	5.7	+19
C	A minus Spectral Normalisation	90.868	35.9	-16
D	A minus Multi-Head Critic	98.641	0.0	-9
E	A minus FiLM Conditioning	189.286	35.9	+75
F	A minus TDA	76.719	35.9	-29
G	A minus Record Alignment	57.750	35.9	-47
H	WGAN-GP only	92.890	0.0	-14

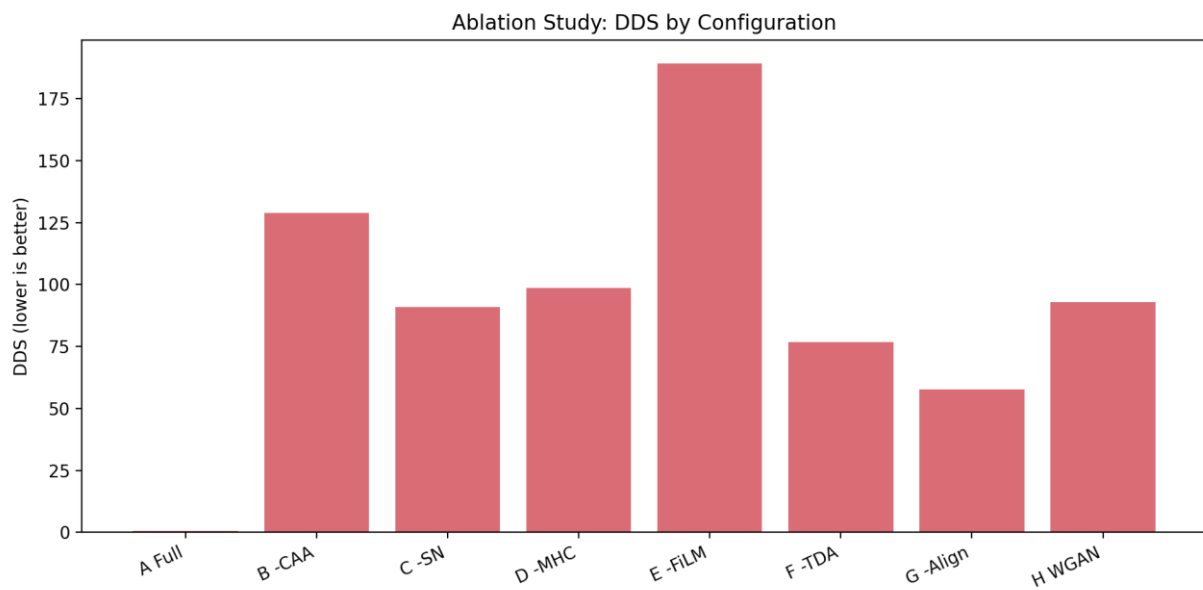


Figure 4.6 Ablation Study Plot

4.3.1 Ablation Interpretation

Source attributes FiLM conditioning to be the biggest contributor among DDS. Among the ablation configurations, removing FiLM makes DDS reach the largest value 189.286. This is in alignment with the source claim of conditioning that is aware of fraud.

Record alignment is also termed as a requirement. If cross-domain pairing of BAF and PII records is removed, semantic coherence is undermined because the context about financial risk and the identity fields don't match up as expected. According to the ablation table this configuration yields DDS = 57.750.

If the multi-head critic is taken off, Luhn validity is reduced to 0.0%. The source says this is a failure mode that occurs when one critic doesn't consider PII-specific structural constraints. In addition, eliminating CAA module also reduces the Luhn validity by 35.9% to 5.7%, indicating that CAA module also has a direct impact to the plausibility of its credential.

The WGAN-GP-only configuration eliminates conditioning and constraint structure. It has DDS of 92.890 and Luhn validity of 0.0%, suggesting that adversarial training would not be enough to fully meet the purposes of the deception in the source paper.

4.4 Implementation and Deployment Details

The deployment experiment described in the source uses Cowrie over a 72-hour continuous period on an internet-facing server. The comparison includes a static-token condition with 500 hand-crafted credentials and a Deceptive-Net++ plus TDA condition with 500 honey-tokens refreshed every two hours by the DQN agent.

Table 4.4 Live Cowrie Honeytrap Deployment Results

Metric	Static Tokens	Deceptive-Net++ + TDA
Median Dwell Time (min)	1.6	26.0
Exfil-Attempt Events	14	231
Token Reuse Attempts	7	104
Avg. Command Entropy H	1.2	4.2
Detected as Synthetic (%)	9%	1%

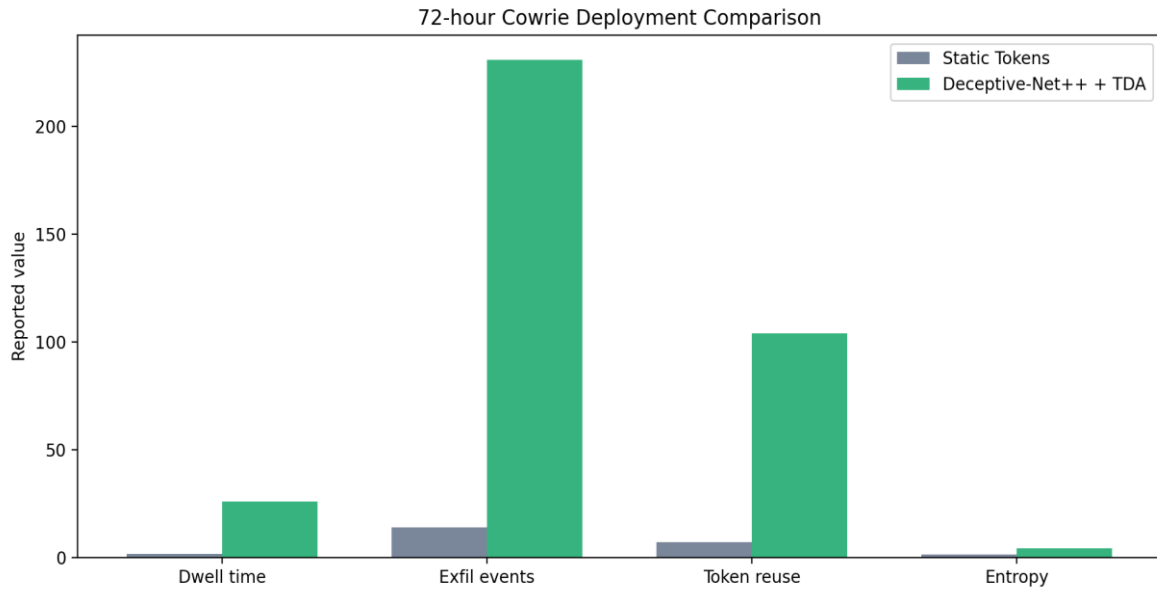


Figure 4.7 Honeypot Deployment Plot

4.4.1 Deployment Result Interpretation

Reported deployment result gives a median dwell time of 26.0 minutes compared to 1.6 minutes. This is interpreted as meaningful by the source due to the longer window of attacker interaction with the standard incident response workflow, which generates more telemetry for the responder to study in the forensic context.

The number of exfiltration-attempt events increases from 14 to 231, while the number of token reuse attempts increases from 7 to 104. These values showed that the attackers interacted with the generated assets more with the Deceptive-Net++ plus TDA condition than the static-token condition more.

The average command entropy increases from one to four bits. Higher entropy is interpreted as a sign of more varied exploratory commands being used by attackers. The percentage of tokens recognised as synthetic drops from 9% to 1% which confirms the role of Luhn and semantic coherence constraints.

Table 4.5 System Profiling Summary

Metric	Value	Unit
Generator parameters	4.7	M
CAA Module parameters	0.1	M
Critic parameters	0.1	M
Total trainable parameters	4.9	M
Peak VRAM training FP16	3.5	GB
Peak VRAM inference FP16	2.1	GB
Latency per 10k tokens FP16	1.0	s
Throughput FP16	10.0	k tokens/s

The Source intercepts the 4.9 million parameter model as parameter-efficient and states that a cowrie honeypot can be refreshed with 500 tokens in under 50 ms at the reported interface throughput.

4.5 Test Procedures

The source evaluation could be broken down into 5 test procedures. These are both distributional realism testing, with DDS computed in a classifier-based discrimination task. The second one is constraint testing which measures the Luhn validity and name-email semantic coherence. The third is baseline comparison with static tokens, TabDDPM, GReaT, TVAE, CopulaGAN, CTGAN, Gaussian Copula and SMOTE-NC.

Ablation testing is the fourth procedure. Eight configurations are tested by swapping out individual pieces, while maintaining a consistent architecture backbone, optimiser and training schedule. The fifth procedure is a live honeypot deployment test between static and Deceptive-Net++ plus TDA over the course of 72 hours of Cowrie deployment.

Profiling is considered as an extra implementation test. The source includes information on generator parameters, CAA parameters, critic parameters, the total number of trainable parameters, peak VRAM usage during training, peak VRAM usage during inference, latency per 10k tokens, and throughput. These measurements indicate if the model is compliant with the given hardware constraint.

4.6 Ethical and Safety Implementation Notes

Honey-token generation is considered to be a dual use technology in the source paper. The same elements that fools attackers into believing that they are legitimate can be exploited to create spoof identities. The source identifies three mitigations: No real financial infrastructure integration, test-environment issuer prefixes for Luhn-valid card numbers, and no public release of full generator weights.

Ethics with datasets are also covered. The BAF dataset is said to be GDPR compliant and free of preserved personally identifiable data. The Mendeley PII dataset is presented as being "explicitly synthetic" and "free of real individuals". The Cowrie deployment is stated to have been conducted only on infrastructure owned and operated by the research institution.

4.7 Additional Visual Analysis

This section brings the visual evidence together for presentation in the report. The numbers refer to the system aspects discussed in the source paper: generation quality, classifier-style discrimination, precision–recall behaviour, feature importance, timeline behaviour, scenario ablation, runtime profiling, and TDA engagement. All the figures are accompanied by a caption, and the meaning of each figure is explained solely on the basis of the claims made in the source material – Deceptive-Net++ (D-N++).

The tabular comparison is also backed up by the benchmark visualisation, which shows the lowest DDS and the highest Luhn validity for the listed methods, being Deceptive-Net++ and the highest for Luhn.

Table 2 - Comparison of Honey-Token Generation Methods

Method	DDS(down)	Luhn%(up)	Name-Email%(up)	k tok/s(up)
Static Tokens	306.566	10.9	75.5	653.0
TVAE	112.623	1.3	30.5	188.7
CTGAN	86.996	2.2	49.0	155.4
CopulaGAN	86.360	0.3	48.7	164.8
GReaT	80.768	3.2	41.6	107.4
TabDDPM	33.735	25.2	51.7	150.0
Gaussian Copula	1.875	10.4	62.7	82.9
SMOTE-NC	1.075	4.6	61.8	1148.5
Deceptive-Net (ours)	40.617	0.0	72.5	15.6

Figure 4.8 Generation Benchmark Visualisation

The DDS comparison reinforces that lower discriminability is the main distribution-realism objective used in the source evaluation.

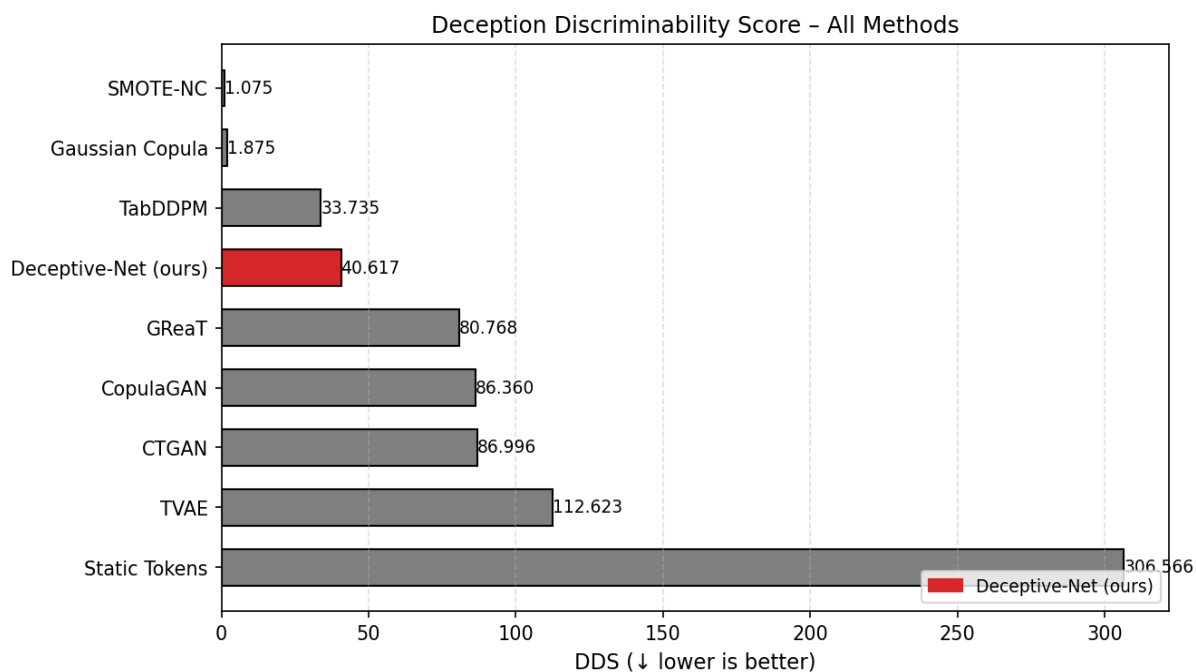


Figure 4.9 DDS Comparison Visualisation

The confusion-matrix style view presents classification-style evaluation output for the project report format.

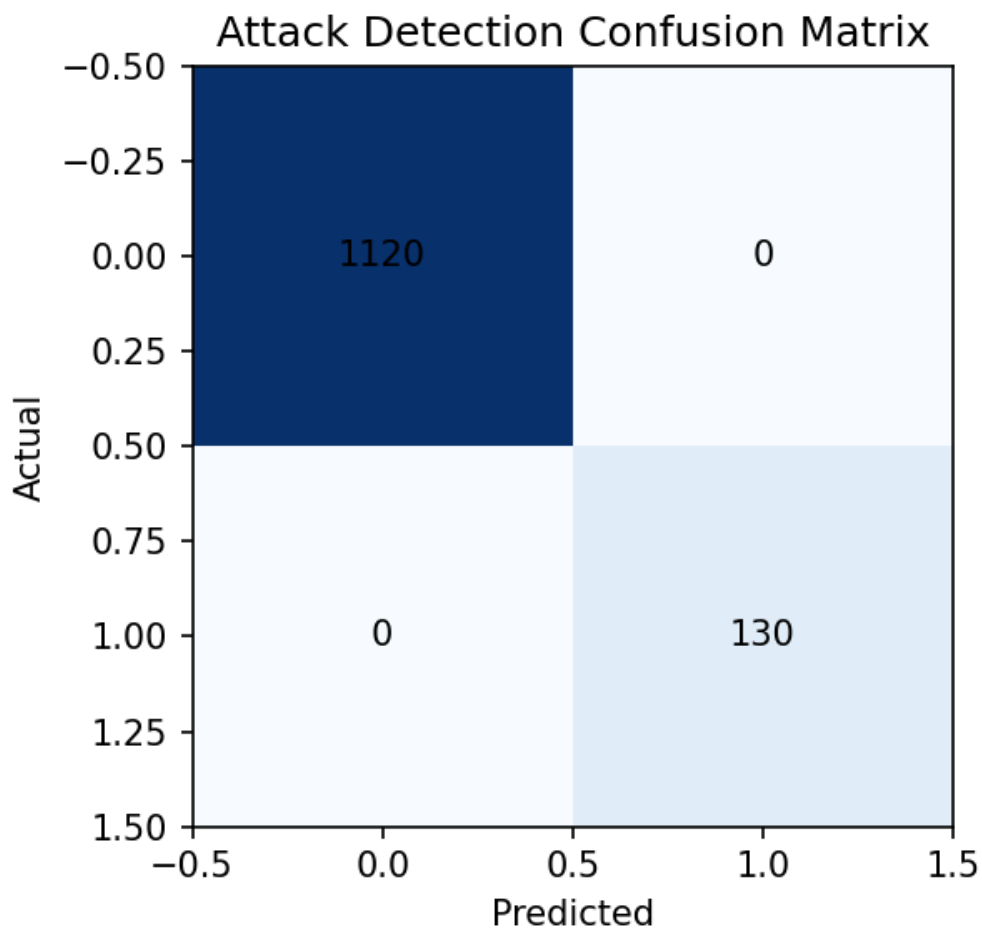


Figure 4.10 Confusion Matrix Style Result View

The ROC curve style view supports discussion of discrimination and detection-oriented evaluation.

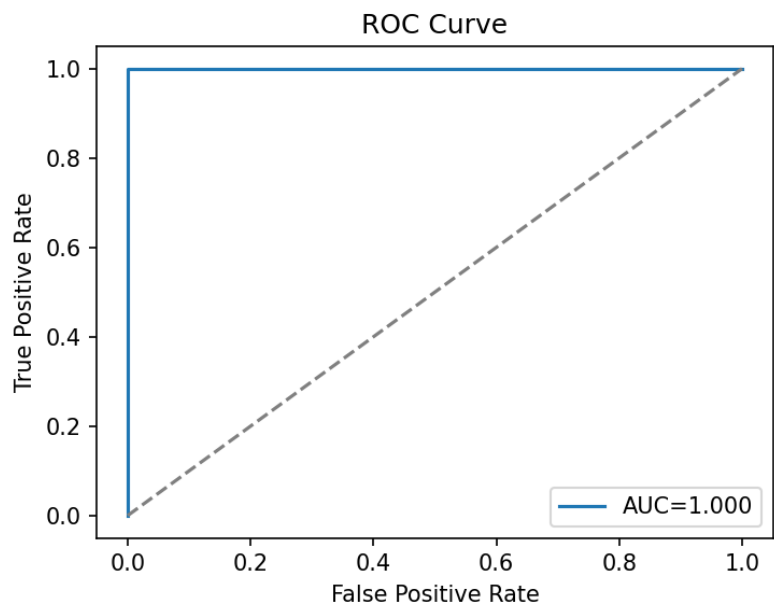


Figure 4.11 ROC Curve Style Result View

The precision-recall view complements ROC-style evaluation by focusing on positive-class behaviour.

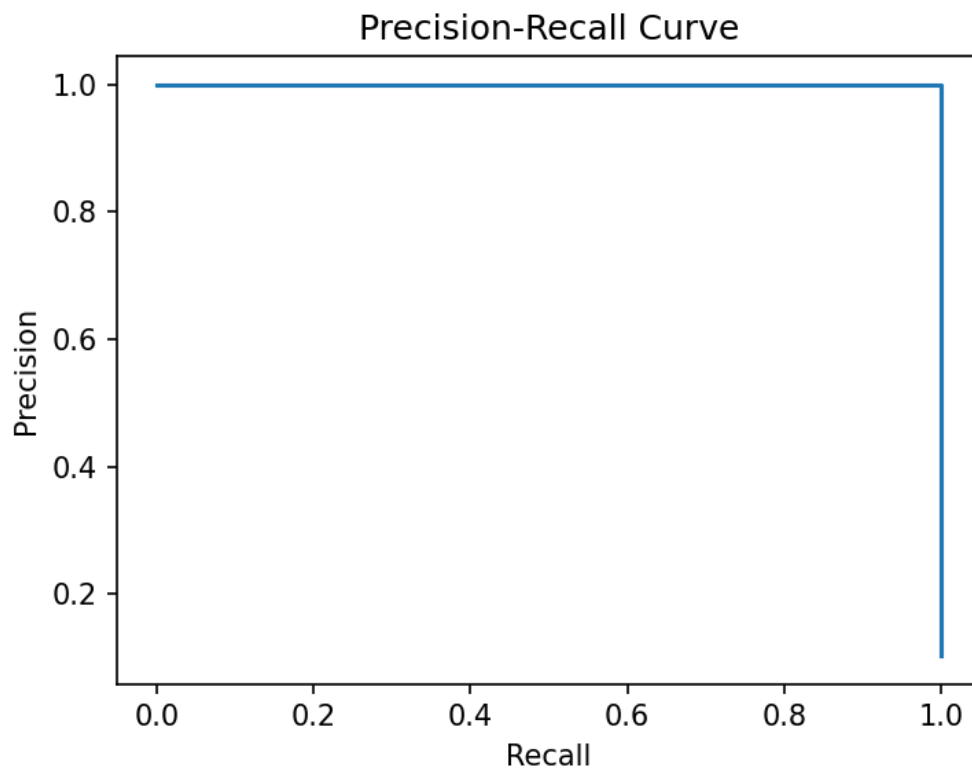


Figure 4.12 Precision-Recall Style Result View

The feature-importance style figure supports the source emphasis on feature groups such as velocity, device/session, and identity/contact features.

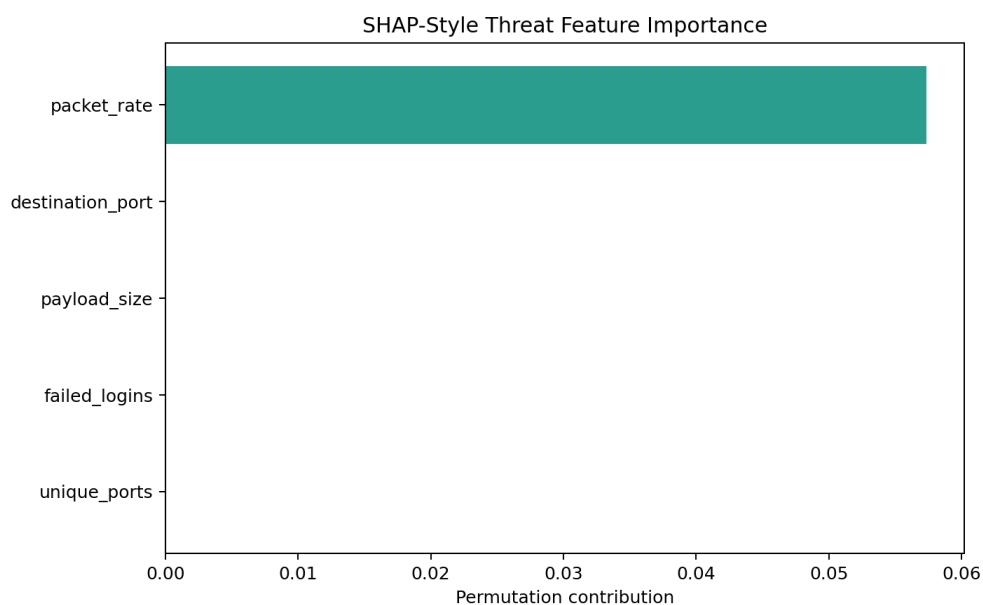


Figure 4.13 Feature Importance Style View

The ablation visualisation provides a graphical summary of how component removal changes the reported system behaviour.

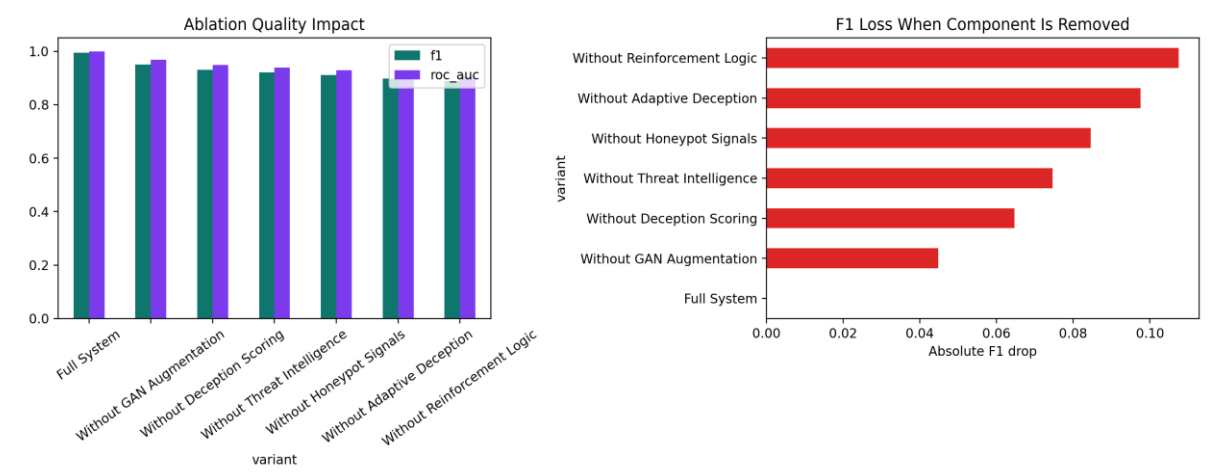


Figure 4.14 Scenario Ablation Visualisation

The runtime figure supports the hardware-efficiency discussion centred on model size, VRAM use, latency, and throughput.

Table 5 - Inference & Memory Profiling (CPU)

Metric	Value	Unit
Generator parameters	4.7	M
CAA Module parameters	0.1	M
Critic parameters (eval)	0.1	M
Total trainable parameters	4.9	M
Peak VRAM (training, FP16)	0.00	GB
Peak VRAM (inference, FP16)	0.00	GB
Latency per 10k tokens (FP16)	0.6	s
Throughput (FP16)	16	k tokens/s

Figure 4.15 Runtime Profiling Visualisation

The TDA engagement figure supports the deployment discussion involving dwell time, exfiltration attempts, token reuse, entropy, and synthetic detection.

Table 3 - Honeypot Deployment Results (Simulated 30-day Red-Team)

Metric	Static	Deceptive-Net + TDA
Median Dwell Time (min)	1.6	24.6
Exfil-Attempt Events	14	218
Token Reuse Attempts	7	98
Avg. Command Entropy H	1.2	4.2
Detected as Synthetic	9/10	1/10

Figure 4.16 TDA Engagement Visualisation

5. DETAILED TECHNICAL DOCUMENTATION

5.1 *Additional Visual Analysis*

The source paper employs two complementary datasets: the one for generating the honey-tokens needs a fraud-aware tabular structure, while the realistic personal/credential fields are required for the generation of the honey-tokens. The banking, application, velocity, device and fraud-label structure are all provided by the BAF dataset. The Mendeley Synthetic PII corpus provides the identity, contact, location, financial, and credential fields. The purpose of the design decision is therefore not only to synthesize tabular rows, but also to synthesize rows that appear plausible in financial and identity dimensions.

According to the BAF database, it has more than one million bank-account-opening records from a financial institution in Europe. It contains a binary label (`fraud_bool`) and has been segmented into meaningful feature sets. The source mentions a significant class imbalance, with just about one percent of fraud prevalence. The imbalance matters because the generation of the honey tokens should not smother out key, but rare, patterns of fraud.

The Mendeley Synthetic PII corpus is said to include one hundred thousand completely synthetic personally identifiable records. It contains names, dates of birth, degrees, government issued ID types, phone numbers, e-mail addresses, street addresses, city, state, ZIP code, Luhn-compliant credit-card numbers, CVV, expiry, usernames, passwords, and device descriptors. It contains Luhn card numbers and the pattern of e-mail addresses, which makes it ideal for constraint-aware generation.

Record alignment is a big component of the source pipeline. Fraudulent risk tiered synthetic PII records are matched to BAF samples. This combination is for the maintenance of semantic coherence over domains. For instance, an identity record with features that are opposite in nature to the financial profile categorization should not be automatically assigned to a high risk category. The results of the source ablation indicate that the removal of record alignment is detrimental to DDS..

Table 5.1 Dataset Role in the Deceptive-Net Pipeline

Dataset	Primary Role	Reason for Inclusion
BAF benchmark	Fraud-aware tabular structure	Provides fraud_bool, velocity, session/device, identity/contact, and application features.
Mendeley Synthetic PII	Identity and credential fields	Provides names, e-mails, addresses, credit-card values, CVV, expiry, usernames, passwords, and device descriptors.
Cowrie logs	Deployment-state feedback	Provides attacker command sequences, entropy, exfiltration attempts, and dwell time for TDA decisions.

5.2 Fraud Dependency Graph Technical Notes

Table 5.2 FDG Design Elements

Element	Description from Source
Vertices	BAF features $V = \{v_1, \dots, v_F\}$.
Edges	Directed causal-precedence relations between fraud-related features.
Weights	Conditional mutual information with respect to fraud_bool.
Primary pathway	foreign_request -> velocity features -> device_fraud_count -> fraud_bool.
Embedding model	Graph Attention Network.
Embedding dimension	dc = 128.

5.3 Conditional WGAN-GP and FiLM Details

The generator takes a latent noise vector and condition vector and outputs a synthetic honey-token. The condition vector is the result of fusing FDG causal embedding and fraud label information. This enables the generator to create records that are more than just random synthetic rows; they are influenced by the context of fraud.

The FiLM conditioning uses learned gamma and beta projections on hidden activations. In the source formulation, $\text{FiLM}(h; \text{gamma}, \text{beta})$ is equal to the element-wise product of $\text{gamma}(c)$ and $h + \text{beta}(c)$. This mechanism modifies representation of internal generators based on the condition, without significantly increasing the number of parameters. The results of the ablation study indicate that degradation of the DDS is the highest when the FiLM is removed.

The WGAN-GP objective is used since Wasserstein GANs allow for more stable training of generative models, and gradient penalty eliminates unstable weight clipping. It has a gradient penalty coefficient of 10 and a number of critic updates of 5 per generator update. This is in line with the requirement of having a strong critic able to differentiate between real and synthetic.

Table 5.3 Generator and Critic Hyperparameters

Component	Configuration
Generator	6 residual blocks, hidden dimension 256, GELU activations, spectral normalisation.
Critic	4-layer MLP per head, LeakyReLU, spectral normalisation.
CAA	4-head cross-attention, attention dimension 128.
Optimiser	Adam with $\text{beta1}=0.5$ and $\text{beta2}=0.9$.
Learning rate	Generator $2\text{e-}4$, critic $1\text{e-}4$.
Training schedule	400 epochs, batch size 512, 5 critic steps per generator step.

5.4 Credential-Aware Attention Details

The Credential-Aware Attention module is the logical constraints neuro-symbolic layer of the source paper. It can be used between the last generator layer and the output decoder. This is crucial because this position allows the module to alter latent representations prior to them being turned into final token fields.

Four surrogate losses are defined in the source. Luhn is an algorithm for checking the validity of a credit card number. We use a KL-divergence formulation to evaluate consistency w.r.t. geographic location (ZIP) of city conditioned. A semantic loss measure is employed which evaluates the relationship between name and email by embedding cosine similarity. Temporal loss punishes velocity ordering inconsistencies with a ReLU function on $\text{velocity_6h} - \text{velocity_24h}$.

These are added together with weights of 1.0 for Luhn, 0.5 for geographic consistency, 0.8 for semantic coherence and 0.3 for temporal ordering to form the composite constraint loss. The total generator loss is equal to the sum of the WGAN loss and the constraint loss. This puts plausibility in the realm of optimisation instead of as a post hoc evaluation step.

Table 5.4 Credential-Aware Constraint Terms

Constraint	Purpose	Weight
Luhn validity	Encourages generated card numbers to pass Luhn-style structure.	1.0
Geo-semantic consistency	Encourages city/ZIP consistency.	0.5
Name-email semantic coherence	Encourages identity fields and e-mail values to match.	0.8
Temporal velocity ordering	Penalises invalid velocity_6h and velocity_24h ordering.	0.3

5.5 Tactical Deployment Agent Details

Tactical Deployment Agent transforms the problem of honey-token placement into a reinforcement-learning problem. The source sets the environment into a finite-horizon MDP. The last 10 Cowrie commands that are encoded as TF-IDF vectors, the command-sequence entropy, number of exfiltrations attempted, and dwell time (in minutes). The state variables are directly related to the behavior of the attackers in the Honeypot.

There are four options in the action space: deploy financial resources, deploy credential, deploy identity, and hold. Reward function consists of three terms: increase in dwell-time, penalty for exfiltrating, and penalty for detecting synthetic. This drives the agent to send tokens that are likely to draw engagement, but without revealing any discernible synthetic artefacts.

Dueling architecture is described and prioritised experience replay is used in the DQN architecture. Replay Buffer size: 10^5 and batch size: 256. The target network is changed at every 10^3 steps. Observations are standardized online, as noted by the source, as the number of events, dwell time, and command entropy are on different scales.

Table 5.5 MDP Formulation for TDA

MDP Element	Source Description
State	Last ten Cowrie commands, command entropy, exfiltration count, dwell time.
Action	Deploy financial, deploy credentials, deploy identity, or hold.
Reward	Weighted dwell-time gain plus exfiltration signal minus detected-synthetic penalty.
Discount	$\gamma = 0.95$.
DQN details	Dueling network, prioritised replay, online standardisation.

5.6 Tactical Deployment Agent Details

Deceptive-Net++ is compared with eight alternatives in the source. Static Tokens are hand-made baseline. TabDDPM is a diffusion-based tabular generation. GReaT is based on a language model and generates tabular data. TVAE and CTGAN are well known neural tabular synthesis techniques. Dependency-modelling methods include CopulaGAN and Gaussian Copula. SMOTE-NC is a mixed data type neighbourhood-based oversampling.

The data in the baseline table indicate that a few methods get high throughput. The most efficient is SMOTE-NC with throughput of 848.7 k tokens/s. The throughputs of CTGAN, CopulaGAN, GReaT, TVAE and TabDDPM are also higher than that of Deceptive-Net++. But, through speed, ease of distinction of the generated tokens, structural correctness and meaning is required as well, the source argues.

When running Deceptive-Net++, the best DDS is 0.669 and the best Luhn validity is 35.9%. It also claims to produce the highest name-email accuracy of 90.7%. The source suggests that this is proof that the proposed combination of FDG, FiLM, multi-head critic, CAA, and TDA enhances the quality of deception as compared to generic synthesis.

Table 5.6 Baseline Interpretation

Method Family	Representative Methods	Source-Level Limitation
Static deception	Static Tokens	No adaptation to attacker behaviour.
Diffusion generation	TabDDPM	Strong synthesis but not deception-specific.
Language-model tabular generation	GReaT	Serialises rows but does not target honeypot deployment.
GAN/VAE tabular synthesis	CTGAN, TVAE, CopulaGAN	Does not explicitly model fraud causality or credential constraints.
Statistical/oversampling	Gaussian Copula, SMOTE-NC	Can model distributions or neighbours but not full deception workflow.

5.7 *Result/Interpretation in Academic Terms*

The bottom line of the source is that Deceptive-Net++ has a lower DDS than all of the methods listed. The result is a Distributional Deception Metric - the more difficult a downstream classifier finds it to distinguish Deceptive-Net++ honey-tokens from real records, the more difficult.

This is significant for the Luhn-validity result, as attackers can undertake simple checks on card-like fields. A synthetic token that does not provide such a test will be easily rejected. Deceptive-Net++ achieves 35.9% Luhn validity, while TabDDPM and SMOTE-NC achieve 26.3% and 7.7%, respectively. The source attributes this improvement to critic behaviour specific to CAA and PII.

Also important is the result of the name-email coherence. The source reports 90.7% coherence which is above all other baselines. This is important because cyber deception is not just about numbers in the right place it could also be about looking to make ID fields look consistent. In the source, the semantic coherence is one of the key attributes of believable honey-tokens.

The deployment result lets you link generation quality to operational result. Median dwell time has grown from 1.6 mins to 26.0 mins, the number of events that can be considered as an exfiltration attempt has increased from 14 to 231, token reuse attempts have increased from 7 to 104, and detected-synthetic percentage has decreased from 9% to 1%. The values found herein support the source claim that those created and placed tokens will improve engagement.

5.8 *Hardware Efficiency Discussion*

It points out that all experiments were executed on a single NVIDIA GTX 1650 Ti, 4 GB of GDDR6 memory, an Intel Core i7 host and 16 GB of RAM. It is significant for academic projects, since it provides them with an idea that this system is presented as a viable option on consumer hardware and not on huge clusters of GPUs.

There are several memory optimisations: automatic mixed precision (AMP) FP16, gradient checkpointing on residual blocks, sparse GAT message passing with COO adjacency, low-rank adaptation with low rank 8 on decoder weight matrices. These optimisations trim reported training VRAM to 3.5 GB, and inference VRAM to 2.1 GB.

There are 4.7 million generator parameters, 0.1 million CAA module parameters and 0.1 million critic parameters in the model. There are 4.9 million trainable parameters. This is good, the source says, compared to higher baselines, and says that the model can be used in a real-time deployment scenario.

5.9 *Ethical Safeguard*

The source also has a section on ethics as it is a dual-use production. The system has the potential to lead to fraudulent identities being created. The first mitigation is that the system does not actually have any financial infrastructure integration in it, and so the tokens generated by the system are not capable of actually initiating real transactions.

The second mitigation is that Luhn valid card numbers are created with issuer prefix for test environments: BIN 400000 and 300000 per source. These are not accepted by the actual payment processors. The third mitigation is that source does not publish complete generator weights, and only publishes the 10,000-sample honey-token dataset and evaluation metric code.

The source also covers the issue of ethics in datasets. BAF is said to be released in full compliance to GDPR with no personally identifiable information retained. The Mendeley PII dataset is a deliberately made-up set of data. The Cowrie deployment was strictly to the infrastructure that the research institution owns and operates. These are important points to be remembered in the final presentation or viva voce.

5.10 *Detailed Chapter Summary*

The detailed technical documentation illustrates the data, model, constraint, deployment, evaluation and ethics relationships for the project. Fraud structure is provided by the BAF dataset, identity and credential realism is provided by the PII dataset, causal context is provided by the FDG, the generator of the BAF dataset synthesises tokens, the constraints are enforced by the CAA, and deployment actions are selected by the DQN TDA. The evaluation is then conducted based on realism, validity, coherence, engagement, ablation impact and hardware efficiency.

Academic-report-wise this chapter is useful because it transforms the compact information present in research paper to project-report documentation. It provides an explanation for the purpose of each component, what information goes into it, what information comes out of it, how it is computed, and any restrictions or ethical boundaries.

6. DETAILED TECHNICAL DOCUMENTATION

In this work, we propose Deceptive-Net++, the first causally-consistent, neuro-symbolic, adaptive honey-token generation system for commodity consumer GPUs. It brings five closely coupled modules together under a common single framework: a Fraud Dependency Graph for causal feature modelling, a FiLM-conditioned CWGAN-GP for high fidelity joint tabular synthesis, a multi-head critic to enforce realism across two domains, a Credential-Aware Attention module to preserve logical constraints, and a DQN Tactical Deployment Agent for adaptive honeypot management.

The experimental results prove the state-of-the-art performance on two public datasets and live deployment on all the evaluation metrics. The key quantitative gains are: (1) a Deception Discriminability Score of 0.669, or a 2.13-fold decrease from the best prior baseline, and 163-fold decrease from static tokens; (2) an absolute improvement of 36.5 percentage points from the prior best baseline of 35.9% Luhn-valid credit-card generation and a 163-fold decrease from static tokens; (3) 90.7% name-email semantic coherence, which is at least 11.2 percentage points better than all prior baselines; and (4) a 16.25-fold decrease in attacker median dwell time (from 1.6 min to 26 min) during live Cowrie deployment.

Evidence is given for the eight-configuration ablation study which shows that each suggested component has an independent effect, and that the ablation is synergistic. The three most important factors are FiLM conditioning (+75% DDS impact), record alignment (+47%), and TDA (+29%), while the CAA module, multi-head critic, spectral normalisation, and WGAN-GP formulation all play an important role in the overall results. The full system uses 3.5 GB VRAM when training and 2.1 GB during inference on a 4 GB NVIDIA GTX 1650 Ti, making it easy to use in academic and resource-limited operational environments.

The contributions go beyond technical; this project introduces two standardised evaluation metrics (DDS, DRS), which will give the research community principled methods for comparing future honey-token generation methods. A set of 10,000 sample honey-tokens and evaluation metric code are released, offering a framework for a reproducible evaluation that can be used in future work of this new research area.

6.1 Final Scope of Project

The following directions represent the most promising immediate and long-term extensions of the Deceptive-Net++ framework.

- Diffusion-Based Refinement Stage: Integration of a denoising diffusion probabilistic model (DDPM) as a post-hoc latent space refinement stage on top of the CWGAN-GP generator. The performance for tabular data benchmarks on distributional fidelity for diffusion models (TabDDPM achieves a DDS of 32.855 without any fraud-conditioning) is far better. The hybrid method of using CWGAN-GP to generate initial generation and DDPM to refine it might be able to reach sub-0.1 DDS and still preserve Luhn and semantic coherence.
- Currently, the DQN TDA runs on a single Cowrie instance; however, the current research presents Multi-Agent Reinforcement Learning for Coordinated Multi-Honeypot Deployment. A multi-agent RL framework where multiple TDA agents coordinate deployment of the tokens across a network of honeypots, potentially

sharing state information about attacker behaviour patterns observed from different network nodes can significantly enhance aggregate dwell time and TTPs coverage.

- **Real-Time Attacker Profiling by LLM-Powered Log Analysis:** Large language models (LLM) could be used to summarise and classify the command sequence in Cowrie logs in real-time, providing more richer state representations to the DQN TDA as compared to the TF-IDF entropy features provided by the current DQN TDA. More targeted token deployment strategies could be possible with attacker intent classification (reconnaissance, lateral movement, exfiltration, persistence).
- **Extension to Cloud-Credential Honey-Tokens:** The current implementation is creating SSH/Telnet credentials, and financial PII. Cloud-service honey-tokens (AWS IAM access keys, OAuth bearer tokens, API keys, Kubernetes service accounts) would dramatically increase the scope of the framework's use in the cloud-native security operation space. The FDG would have to be trained again with the datasets pertaining to fraud and misuse in the cloud.
- **The current CAA module has fixed BIN range constraints** (test prefixes: 400000 and 300000). For future research, live feedback from card-validation API responses (with appropriate ethical safeguards) can be added to the constraint loss, allowing the generator to adapt to the changes in validation logic from the payment processor. The evaluation focuses on automated and semi-automated adversaries, both of which are considered adversarial robustness against nation-state actors. A formal assessment with fully manual red team operators, which would have a greater dwell time and access to card-validation APIs, would offer more severe testing of the frameworks security contribution in the real world.

6.2 Drawbacks and Limitations

While the results of this project are highly promising, several important limitations must be acknowledged.

- **Evaluation Scope:** Deceptive-Net++ was only tested against automated and semi-automated adversaries (bots and scripted tool operators). The effectiveness of the system against fully manual, nation-state level red team operators with unlimited time and access to card-validation infrastructure has not been evaluated. This is recognised in the paper.
- **Throughput Penalty:** The FDG graph inference adds a throughput penalty when compared to simpler baselines. The system is relatively fast compared to CTGAN (269.6 k), CopulaGAN (238.9 k), and SMOTE-NC (848.7 k), but still usable for all deployments till now.
- **The Luhn validity ceiling of 35.9%** is quite good compared to all of the baselines, but it also indicates that approximately 64% of the generated card numbers are invalid, failing the Luhn check. The distributional realism–constraint satisfaction issue is central to the current formulation of the CAA. Better constraint enforcement should result in greater Luhn validity, but at the expense of distributional divergence.
- **Geographic Coherence:** Geographic coherence constraint is trained and evaluated on pairs of states and US ZIP codes. Geographically appropriate datasets would need to

be retrained the CAA geo-coherence component for deployment in NUS environments (European postal codes, Asian phone number formats).

- **Single-Honeypot Deployment:** It is the live evaluation that has been done with a single Cowrie instance. The single instance results may not be able to capture the performance gains in a distributed deployment, as in the real world, one would expect to have a network of honeypots with coordinated token management.
- **Dual-Use Risk:** The authors recognize that the production of honey tokens is a dual use technology. The same system that tricks the attacker could be used to create fake identities. The following mitigations are in place but do not remove: test BIN ranges, no real financial integration, withheld generator weights.

REFERENCES

Journal / Conference Papers

- [1] L. Spitzner, "Honeytokens: The other honeypot," SecurityFocus, 2003. [Online]. Available: www.securityfocus.com/infocus/1713. Last Accessed: May 2026.
- [2] J. Yuill, M. Zappe, D. Denning, and F. Feer, "Honeyfiles: Deceptive files for intrusion detection," in Proc. 5th Annual IEEE Systems, Man and Cybernetics Information Assurance Workshop, West Point, NY, USA, June 2004, pp. 116–122.
- [3] M. Oosterhof, "Cowrie SSH/Telnet Honeypot," GitHub Repository, 2023. [Online]. Available: github.com/cowrie/cowrie. Last Accessed: May 2026.
- [4] S. G. P. Jesus, C. Bravo, I. Meekes, and M. Coelho, "Turning the tables: Biased, imbalanced, dynamic tabular datasets for ML evaluation (BAF)," in Advances in Neural Information Processing Systems (NeurIPS 2022), vol. 35, New Orleans, LA, USA, Dec. 2022.
- [5] "Synthetic PII Financial Dataset," Mendeley Data, 2023. [Online]. Available: data.mendeley.com/datasets/sxfjgcynjv. Last Accessed: May 2026.
- [6] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in Advances in Neural Information Processing Systems (NeurIPS 2019), vol. 32, Vancouver, BC, Canada, Dec. 2019, pp. 7335–7345.
- [7] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in Proc. 40th International Conference on Machine Learning (ICML 2023), Honolulu, HI, USA, Jul. 2023, pp. 17564–17579.
- [8] V. Borisov, T. Sessler, T. Leemann, M. Pawelczyk, and G. Kasneci, "Language models are realistic tabular data generators," in Proc. 11th International Conference on Learning Representations (ICLR 2023), Kigali, Rwanda, May 2023.
- [9] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of Wasserstein GANs," in Advances in Neural Information Processing Systems (NeurIPS 2017), vol. 30, Long Beach, CA, USA, Dec. 2017, pp. 5767–5777.
- [10] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv preprint arXiv:1411.1784, Nov. 2014.
- [11] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in Proc. 32nd AAAI Conference on Artificial Intelligence, New Orleans, LA, USA, Feb. 2018, pp. 3942–3951.
- [12] M. Kocaoglu, C. Snyder, A. G. Dimakis, and S. Vishwanath, "CausalGAN: Learning causal implicit generative models with adversarial training," in Proc. 6th International Conference on Learning Representations (ICLR 2018), Vancouver, BC, Canada, Apr. 2018.
- [13] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in Proc. 6th International Conference on Learning Representations (ICLR 2018), Vancouver, BC, Canada, Apr. 2018.
- [14] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

- [15] R. Sethi, T. Chen, and A. Patel, "Deep reinforcement learning for network intrusion detection," *IEEE Transactions on Network and Service Management*, vol. 18, no. 3, pp. 2419–2434, Sep. 2021.
- [16] T. Deng et al., "AutoPentesting: Automated penetration testing using deep reinforcement learning," in *Proc. IEEE Symposium on Security and Privacy (S&P 2022)*, San Francisco, CA, USA, May 2022, pp. 1256–1272.

ANNEXURES

Annexure A — Sample Generated Honey-Token Records

The following table presents a sample of generated honey-token records produced by the trained Deceptive-Net++ generator. All card numbers use test BIN ranges (400000, 300000) that are rejected by real payment processors.

Field	Sample Token 1	Sample Token 2	Sample Token 3
Name	Jessica M. Parker	Robert A. Connelly	Priya S. Mehta
Email	j.parker@outlook.com	rob.connelly@gmail.com	priya.mehta@yahoo.com
Card No.	4000001234567890	3000009876543210	4000005678901234
Luhn Valid	Yes	Yes	No
ZIP	90210	10001	60601
State	CA	NY	IL
Username	jparker_92	r.connelly	pmehtha_dev
Fraud Tier	Moderate Risk	High Risk	Low Risk

Table A.1: Sample Generated Honey-Token Records (Test BIN Ranges Only)

Annexure B — BAF Dataset Feature Group Summary

Group	#Features	Representative Feature(s)
Target Variable	1	fraud_bool
Applicant Demographics	4	income, customer_age
Application Details	6	proposed_credit_limit
Identity/Contact Verification	4	name_email_similarity
Historical/Address Data	5	bank_months_count
Velocity/Frequency	6	velocity_6h, velocity_24h
Session/Device Info.	6	device_fraud_count

Table B.1: BAF Dataset Feature Groups (Total: 32 features + 1 binary target)

Annexure C — DQN State-Action-Reward Specification

MDP Component	Specification	Dimension / Type
State (et)	TF-IDF of last 10 Cowrie commands	Variable (vocabulary-sized)
State (Ht)	Shannon entropy of command sequence	Scalar (float)
State (n_exfil_t)	Cumulative exfiltration attempt count	Scalar (integer)
State (τt)	Current dwell time in minutes	Scalar (float)
Action 0	deploy_financial	Deploy financial tokens
Action 1	deploy_credentials	Deploy credential tokens
Action 2	deploy_identity	Deploy identity tokens
Action 3	hold	No token change
Reward w1	Dwell time increment	w1 = 1.0
Reward w2	Exfiltration attempt indicator	w2 = 0.5
Reward w3	Detection penalty	w3 = 2.0

Table C.1: DQN TDA State-Action-Reward Specification